

Reti Neurali

■ Introduzione

- Neuroni Biologici
- Neuroni Artificiali
- Tipologie di Reti

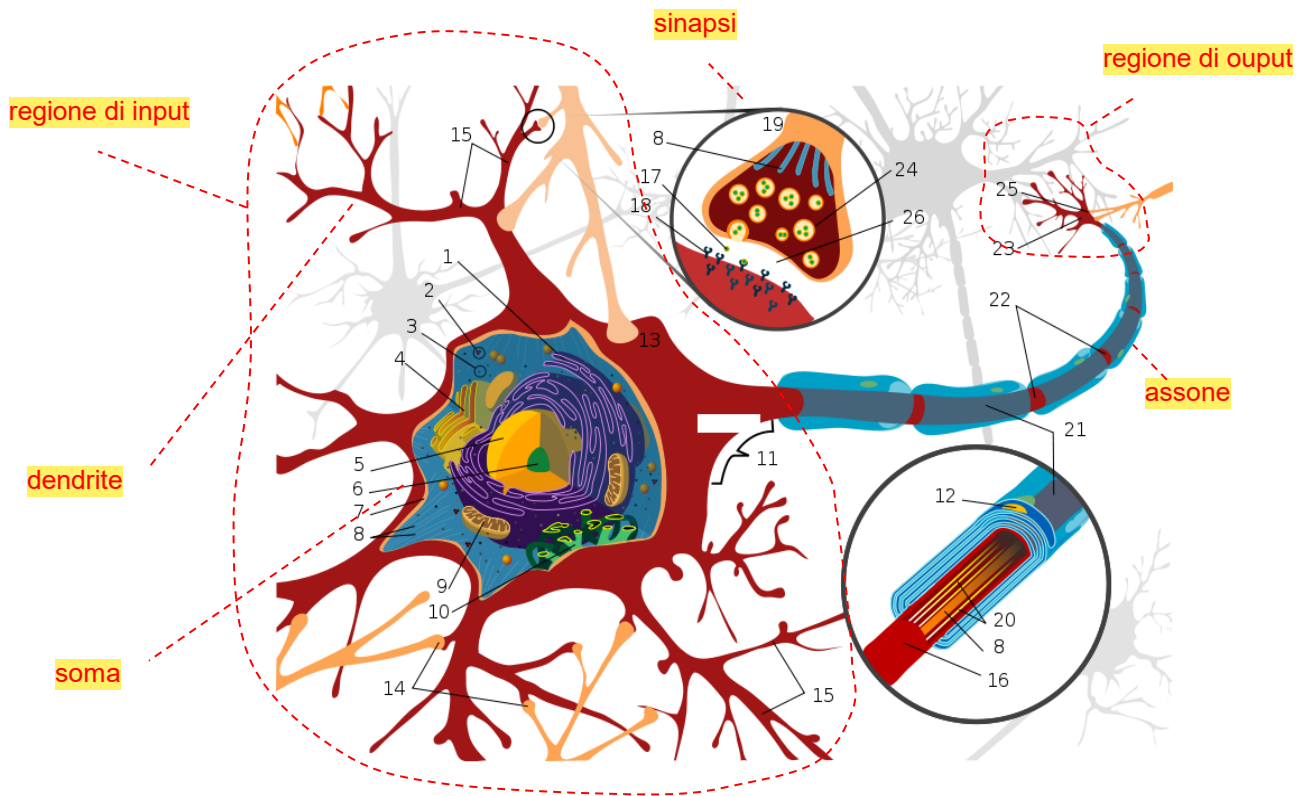
■ Multilayer Perceptron (MLP)

- Forward propagation
- Training: Error Backpropagation
- On-line Backpropagation
- Stochastic Gradient Descent (SGD)

■ Approfondimenti

- Softmax e Cross-Entropy
- Inizializzazione Pesi
- Regolarizzazione (Weight Decay)
- Momentum
- Learning Rate adattativo

Neuroni Biologici



■ I neuroni sono le più importanti **cellule** del sistema nervoso.

Il punto di giunzione dove l'informazione viene trasmessa tra neuroni. Nel neurone artificiale: I Pesi (w_i) della connessione, che determinano l'importanza dell'input.

■ Le connessioni **sinaptiche** o (**sinapsi**) agiscono come porte di collegamento per il passaggio dell'informazione tra neuroni.

Raccolgono i segnali (input) chimici da altri neuroni tramite le sinapsi. Nel neurone artificiale: i vettori di Input x

■ I **dendriti** sono fibre minori che si ramificano a partire dal corpo cellulare del neurone (detto **soma**). Attraverso le sinapsi i dendriti raccolgono **input** da neuroni **afferenti** e li propagano verso il soma.

Integra (somma) i segnali in ingresso. Se la somma supera una soglia, genera un impulso.
Nel neurone artificiale: l'operazione di somma pesata ($\sum w_i x_i$) e l'applicazione della funzione di attivazione.

Trasporta l'impulso elettrico (output) dal soma verso altri neuroni. Nel neurone artificiale: l'Output y che viene passato come input al neurone successivo.

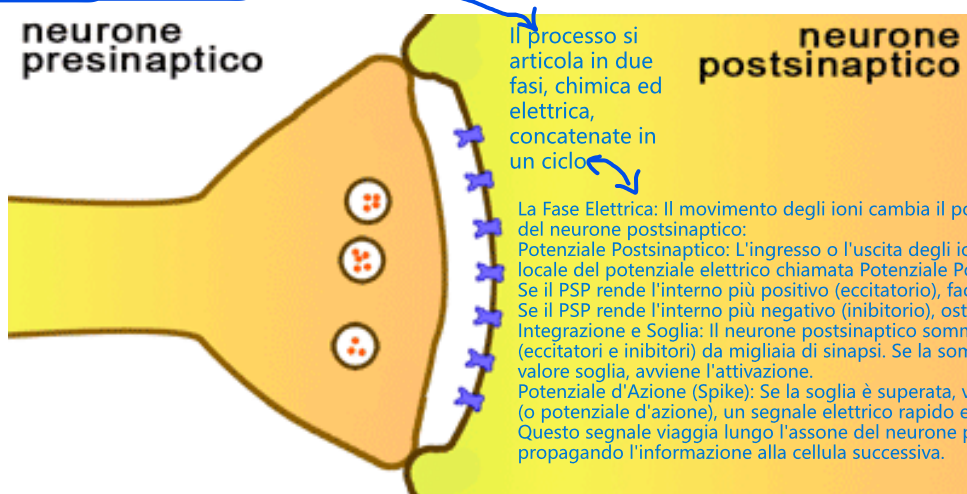
■ L'**assone** è la fibra principale che parte dal soma e si allontana da esso per portare ad altri neuroni (anche distanti) l'**output**.

La Fase Chimica: Questa fase avviene tra il neurone presinaptico (che invia) e il neurone postsinaptico (che riceve).
 Arrivo del Segnale: Un potenziale d'azione (impulso elettrico) arriva al terminale dell'assone del neurone presinaptico. L'arrivo dell'impulso provoca il rilascio di neurotrasmettitori (sostanze chimiche) nella fessura sinaptica, il breve spazio tra i due neuroni.
 Legame con i Recettori: I neurotrasmettitori attraversano il gap e si legano a recettori specifici (spesso canali ionici) sulla membrana del neurone postsinaptico.
 Ingresso di Ioni: Il legame apre i canali ionici, permettendo agli ioni di fluire attraverso la membrana.

Neuroni Biologici (2)

La comunicazione è chimica (rilascio di neurotrasmettitori nel gap sinaptico) e diventa elettrica (generazione di un potenziale d'azione o spike nel neurone post-sinaptico).

Il passaggio delle informazioni attraverso le sinapsi avviene con processi elettro-chimici.



Il neurone presinaptico libera delle sostanze (neurotrasmettitori) che attraversano il breve gap sinaptico e sono captati da appositi recettori (canali ionici) sulla membrana del neurone postsinaptico. L'ingresso di ioni attraverso i canali ionici determina la formazione di una differenza di potenziale tra il corpo del neurone postsinaptico e l'esterno.

il neurone postsinaptico.

Quando questo potenziale supera una certa soglia si produce uno spike (impulso): il neurone propaga un breve segnale elettrico detto potenziale d'azione lungo il proprio assone: questo potenziale determina il rilascio di neurotrasmettitori dalle sinapsi dell'assone.

Questo meccanismo "tutto o niente" (attivazione solo se si supera una soglia) è emulato dalla Funzione di Attivazione nel neurone artificiale, che decide se e quanto un neurone deve "sparare" (attivarsi) in base alla somma pesata dei suoi input.

Il reweighting delle sinapsi (ovvero la modifica della loro efficacia di trasmissione) è direttamente collegato a processi di apprendimento e memoria in accordo con la regola di Hebb.

Hebbian rule: se due neuroni, tra loro connessi da una o più sinapsi, sono ripetutamente attivati simultaneamente allora le sinapsi che li connettono sono rinforzate.

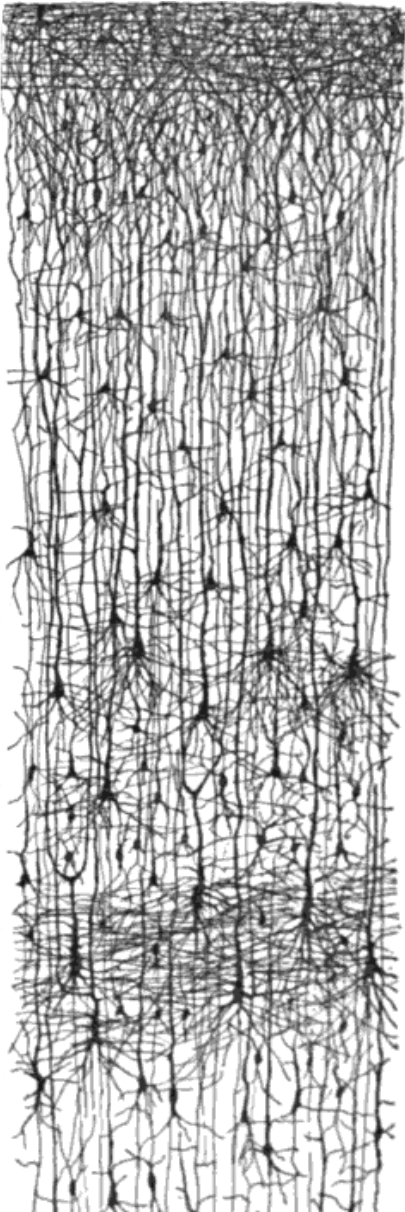
Il "rinforzo" delle sinapsi corrisponde alla modifica dei Pesi (w_i) nella rete neurale artificiale.

Le Deep Neural Networks (DNN) sono modellate direttamente su questa organizzazione a strati. I dati (input) passano attraverso una sequenza di livelli (layers). Ogni strato impara a estrarre caratteristiche (features) di complessità crescente. Questo è particolarmente evidente nelle Reti Neurali Convoluzionali (CNN), dove i primi strati estraggono bordi e texture, e gli strati successivi combinano questi elementi per riconoscere oggetti complessi.

Reti Neurali Biologiche

Il cervello umano contiene circa 100 miliardi di neuroni ciascuno dei quali connesso con circa altri 1000 neuroni (10^{14} sinapsi).

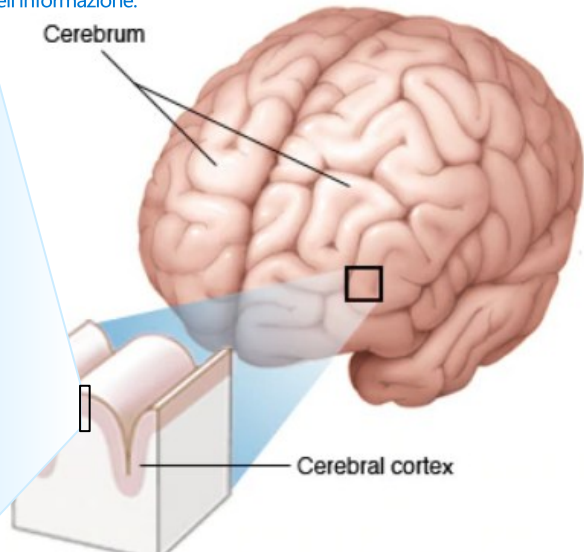
Questi numeri spiegano perché le Deep Neural Networks (DNN) richiedono miliardi di parametri (i pesi w_i) per raggiungere capacità paragonabili, anche se i modelli più grandi di oggi (come i grandi modelli linguistici) hanno solo centinaia di miliardi o trilioni di parametri, ancora lontani dalla densità del cervello. La potenza non risiede solo nel numero di neuroni, ma nell'alta connettività: ogni neurone è in media collegato a circa 1000 altri neuroni. Nelle ANN, la connettività è spesso ridotta (es. nelle Reti Convoluzionali, dove ogni neurone si collega solo a una piccola regione dell'input) o completamente connessa (come nel Perceptron multistrato).



è la sede delle funzioni cognitive superiori (percezione, linguaggio, memoria complessa).

La **corteccia cerebrale** (sede delle funzioni nobili del cervello umano) è uno strato laminare continuo di 2-4 mm, una sorta di lenzuolo che avvolge il nostro cervello formando numerose **circonvoluzioni** per acquisire **maggiore superficie**. Sebbene i neuroni siano disposti in modo «abbastanza» ordinato in **livelli consecutivi**, l'intreccio di **dendriti e assoni** ricorda una **foresta fitta e impenetrabile**.

Strato Laminare e Circonvoluzioni: La corteccia è uno strato sottile (2-4 mm) che si ripiega su sé stesso (circonvoluzioni) per massimizzare la superficie disponibile. Ordine in Livelli: I neuroni non sono disposti a caso, ma in livelli consecutivi (sei strati principali, o lamine). Ogni strato tende a specializzarsi in diverse fasi di elaborazione dell'informazione.



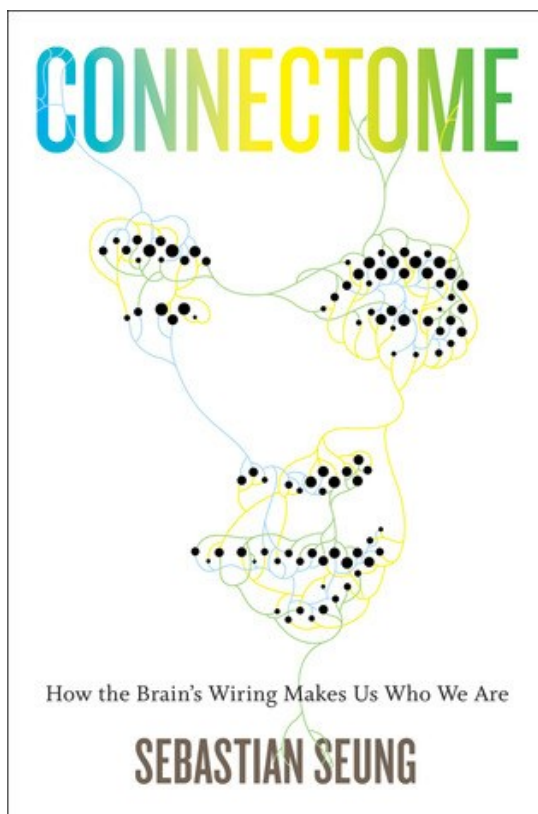
Nonostante l'ordine laminare, la rete di dendriti e assoni che collega i neuroni è estremamente complessa e densa. Questa "foresta" garantisce l'elevata plasticità e la capacità del cervello di re-instradare i segnali e imparare in modi diversi (plasticità sinaptica).

In sintesi, mentre le Reti Neurali Artificiali semplificano notevolmente il neurone (come abbiamo visto con il Perceptron), prendono in prestito l'idea fondamentale della sua organizzazione: elaborazione distribuita di segnali attraverso una struttura a strati altamente interconnessa, dove la potenza deriva dalla combinazione massiccia e dall'organizzazione gerarchica degli elementi semplici.

Connectome

- Il **connectome** è la mappa delle connessioni dei neuroni nel cervello: una sorta di grafo che individua tutti i neuroni e le loro connessioni attraverso le sinapsi.

NO ROAD, NO trail can penetrate this forest. The long and delicate branches of its trees lie everywhere, choking space with their exuberant growth. No sunbeam can fly a path tortuous enough to navigate the narrow spaces between these entangled branches. All the trees of this dark forest grew from 100 billion seeds planted together. And, all in one day, every tree is destined to die.



[Sebastian Seung, 2012]

Connectome

*How the Brain's Wiring
Make Us Who We Are*

Progetto di mappatura
collaborativa ([Eyewire](#))

Mappatura [cervello topo](#)
(da [Blue Brain](#) project)

Connectome (2)

 il DNA stabilisce le fondamenta della rete:

■ Il suo sviluppo pre-natale è guidato dal DNA: La Struttura Iniziale

- I geni sono responsabili della formazione dei neuroni e del loro posizionamento (migrazione).
- Una volta giunti in posizione i neuroni iniziano a estendere le loro ramificazioni (dendriti e assone) e a formare sinapsi. Questi processi sono parzialmente guidati dai geni e in parte random. Questa fase è parzialmente guidata geneticamente e in parte casuale.
- Alla nascita, la creazione e la migrazione sono di fatto completate (solo in poche aree del cervello nuovi neuroni possono essere creati dopo la nascita), mentre la creazione di nuove ramificazioni e sinapsi prosegue ben oltre.

 Dopo la nascita, l'ambiente e l'apprendimento plasmano attivamente la rete esistente (la rete non è statica).

■ Lo sviluppo post-natale (guidato dal DNA e dall'apprendimento).

La rete non è totalmente connessa e le sinapsi non sono create "on-demand" solo quando si impara qualcosa di specifico. La creazione e la modifica sono processi continui e dinamici.

- La rete non è totalmente connessa e le sinapsi non sono create «on-demand» durante i processi di apprendimento.

capacità del cervello di creare nuove connessioni, riattivare percorsi neurali inattivi e riorganizzarsi per compensare perdite o danni.

- I meccanismi di **reconnection** non sono completamente noti ma una delle teorie più accreditate è il Neural Darwinism: ipotizza che nuove sinapsi siano continuamente create random tra neuroni vicini. Quelle successivamente potenziate dal reweighting Hebbiano rimangono attive (contribuendo alla formazione di memorie a lungo termine), mentre quelle non usate sono via via eliminate. I dendriti che perdono gran parte di sinapsi vengono anch'essi eliminati.

- La velocità di formazione di nuove sinapsi è strabiliante nei neonati (oltre mezzo milione al secondo tra i 2 e 4 mesi di età), ma prosegue anche in età adulta.

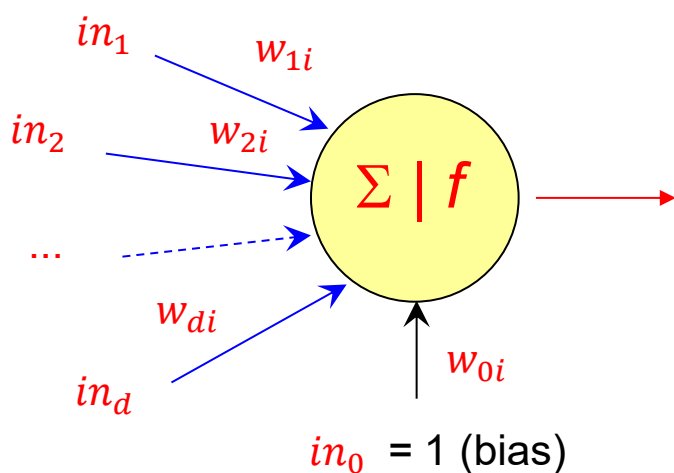
Le sinapsi che vengono utilizzate e rafforzate tramite il reweighting Hebbiano (cioè, l'attivazione simultanea dei neuroni) vengono mantenute e potenziate (sopravvivono e si rafforzano, contribuendo alla memoria a lungo termine).

 Questo è il meccanismo che genera la "varietà" su cui la selezione può agire.

Il suo ruolo è fondamentale: permette di spostare la soglia di attivazione del neurone. Senza il bias, il neurone sarebbe "vincolato" a passare sempre per l'origine; con il bias, invece, può attivarsi anche quando tutti gli input sono piccoli o nulli. In pratica, il bias serve a tarare il punto di lavoro del neurone, proprio come una soglia adattabile.

Neurone Artificiale

- Primo modello del 1943 di McCulloch and Pitts. Con input e output binari era in grado di eseguire computazioni logiche.
- Nel seguito, un neurone (moderno) di indice i : una rete neurale ne contiene molti, dobbiamo distinguerli ...



Il funzionamento di un singolo neurone i può essere scomposto in tre passaggi chiave: Input, Somma Pesata e Attivazione.

$$net_i = \sum_{j=1 \dots d} w_{ji} \cdot in_j + w_{0i}$$

$$out_i = f(net_i)$$

- $in_1, in_2, \dots in_d$ sono i d ingressi che il neurone i riceve da assoni di neuroni afferenti.
- $w_{1i}, w_{2i}, \dots w_{di}$ sono i pesi (weight) che determinano l'efficacia delle connessioni sinaptiche dei dendriti (agiremo su questi valori durante l'apprendimento).
- w_{0i} (detto bias) è un ulteriore peso che si considera collegato a un input fittizio con valore sempre 1; questo peso è utile per «tarare» il punto di lavoro ottimale del neurone.
- net_i è il livello di eccitazione globale del neurone (potenziale interno);
- $f(\cdot)$ è la funzione di attivazione che determina il comportamento del neurone (ovvero il suo output out_i) in funzione del suo livello di eccitazione net_i .

Neurone: funzione di attivazione

■ Nei neuroni biologici, $f(\cdot)$ è una funzione tutto-niente temporizzata: quando net_i supera una certa soglia, il neurone «spara» uno spike (impulso) per poi tornare a riposo.

■ Esistono reti artificiali (denonimate **spiking neural network**) che modellano questo comportamento e processano l'informazione attraverso **treni di impulsi**.

In queste reti l'informazione non è rappresentata da numeri reali, ma da treni di impulsi nel tempo.

sequenza temporale di segnali elettrici discreti

■ È necessaria questa «complicazione»? Oppure codificare l'informazione con impulsi (invece che con **livelli continui**) è solo una questione di risparmio energetico messa a punto dall'evoluzione?

I segnali che attraversano il neurone non sono valori discreti o binari (come 0 o 1, o uno spike), ma possono assumere qualsiasi valore reale all'interno di un certo intervallo.

■ Le reti neurali più comunemente utilizzate operano con **livelli continui** e $f(\cdot)$ è una funzione **non-lineare** ma **continua** e **differenziabile** (quasi ovunque).

■ Perché non-lineare? Se vogliamo che una rete sia in grado di eseguire un mapping (complesso) dell'informazione di input sono necessarie non-linearità.

■ Perché continua e differenziabile? algoritmo utilizzato per addestrare le Reti Neurali. Necessario per la retro-propagazione dell'errore (come scopriremo presto).

■ Una delle funzioni di attivazione più comunemente utilizzata è la **sigmoide** nelle varianti:

1 ■ **standard logistic function** (chiamata semplicemente **sigmoid**)

2 ■ **tangente iperbolica** (**tanh**)

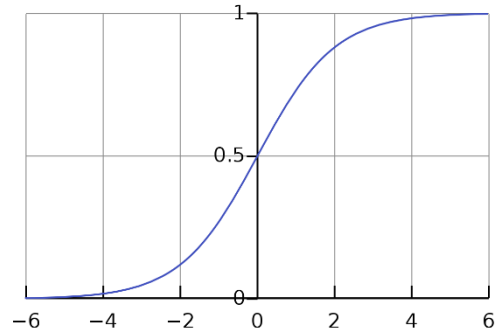
Funzioni di attivazione più recenti (**Relu**, **Elu**) introdotte per reti profonde (deep-learning) sono spiegate in seguito.

Funzione di attivazione: sigmoide

1 ■ Standard logistic function (Sigmoid), (valori in $[0...1]$):

$$f(\text{net}) = \sigma(\text{net}) = \frac{1}{1 + e^{-\text{net}}}$$

Per valori di net molto negativi, $\sigma(\text{net})$ circa 0.
Per valori di net molto positivi, $\sigma(\text{net})$ circa 1.



La derivata:

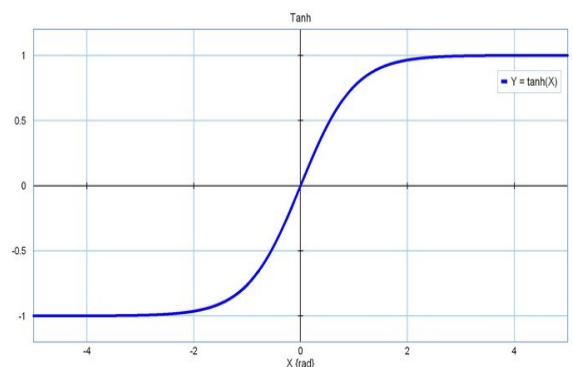
$$\sigma'(\text{net}) = \frac{\partial}{\partial \text{net}} \left(\frac{1}{1 + e^{-\text{net}}} \right) = \frac{e^{-\text{net}}}{(1 + e^{-\text{net}})^2} = \sigma(\text{net})(1 - \sigma(\text{net}))$$

2 ■ Tangente iperbolica (Tanh), (valori in $[-1...1]$):

Può essere ottenuta dalla ~~funzione precedente~~ a seguito di trasformazione di scala ($\times 2$) e traslazione (-1).
sigmoid

- Scala orizzontale: $\sigma(2x)$ rende la curva più ripida.
- Scala verticale: $2\sigma(2x)$ porta l'intervallo da $[0,1]$ a $[0,2]$.
- Traslazione verticale: -1 sposta l'intervallo da $[0,2]$ a $[-1,1]$.

$$f(\text{net}) = \tau(\text{net}) = 2\sigma(2 \cdot \text{net}) - 1$$



La derivata: $\tau'(\text{net}) = 1 - \tau(\text{net})^2$

Tanh è leggermente più complessa di sigmoid ma preferibile (convergenza più rapida), in quanto simmetrica rispetto allo zero.

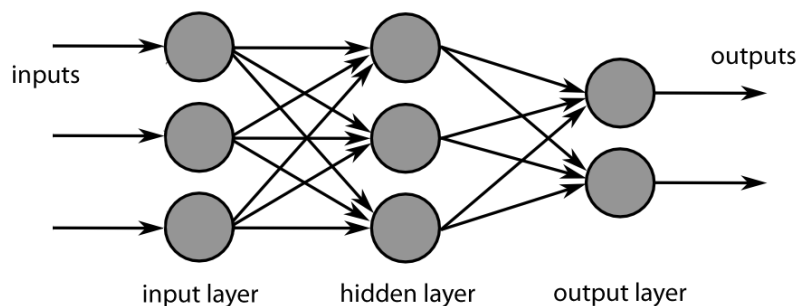
Tipologie di reti neurali

Le reti neurali sono composte da gruppi di neuroni artificiali organizzati in livelli. Tipicamente sono presenti: un livello di **input**, un livello di **output**, e uno o più livelli **intermedi** o **nascosti** (hidden). Ogni livello contiene uno o più neuroni.

- Livello di Input: Riceve i dati iniziali.
- Livello di Output: Fornisce il risultato finale.
- Livelli Hidden: I livelli intermedi tra input e output. È qui che avviene la maggior parte dell'elaborazione e l'estrazione delle caratteristiche complesse.

Il tempo non gioca un ruolo importante nel calcolo interno.

■ **Feedforward**: nelle reti feedforward («alimentazione in avanti») le connessioni collegano i neuroni di un livello con i neuroni di un livello successivo. Non sono consentite connessioni all'indietro o connessioni verso lo stesso livello. È di gran lunga il tipo di rete più utilizzata.



■ **Recurrent**: nelle reti ricorrenti sono previste connessioni di feedback (in genere verso neuroni dello stesso livello, ma anche all'indietro). Questo complica notevolmente il flusso delle informazioni e l'addestramento, richiedendo di considerare il comportamento in più istanti temporali (unfolding in time). D'altro canto queste reti sono più indicate per la gestione di sequenze (es. audio, video, frasi in linguaggio naturale), perché dotate di un **effetto memoria** (di breve termine) che al tempo t rende disponibile l'informazione processata a $t - 1$, $t - 2$, ecc. Un particolare tipo di rete ricorrente è LSTM (Long Short Term Memory).

Le reti ricorrenti introducono connessioni di feedback. Un neurone può ricevere in ingresso anche il proprio output passato o quello di neuroni dello stesso livello o di livelli successivi. Questo cambia completamente il comportamento della rete: l'output a tempo t non dipende solo dall'input corrente, ma anche dallo stato interno accumulato nel tempo. In altre parole, la rete ha una forma di memoria. Quando la rete elabora un elemento della sequenza, conserva informazione su ciò che ha visto prima, rendendola disponibile ai passi temporali successivi. Dal punto di vista matematico, però, questo introduce una forte complessità: per addestrare la rete bisogna "srotolarla nel tempo" (unfolding in time) e applicare una variante del backpropagation che considera più istanti temporali.

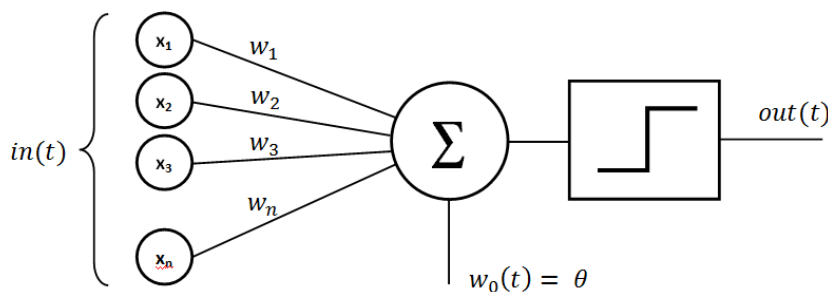
Multilayer Perceptron (MLP)

- Il termine **perceptron** (percettone) deriva dal modello di neurone proposto da Rosenblatt nel 1956.

- Il percettone utilizza una funzione di attivazione lineare a soglia (o **scalino**). Un singolo percettone, o una rete di percettroni a due soli livelli (input e output), può essere addestrato con una semplice regola detta **delta rule** (ispirata alla regola di **Hebb**).

Questo output binario (0 o 1) permetteva di prendere decisioni di classificazione.

che era in grado di trovare i pesi corretti se e solo se i dati di input erano linearmente separabili.



- Una rete a due livelli di percettroni lineari a soglia è in grado di apprendere solo **mapping lineari** e pertanto il numero di funzioni approssimabili è piuttosto limitato.

Il passaggio al MLP ha risolto la limitazione della linearità.

- Un **Multilayer Perceptron (MLP)** è una rete feedforward con **almeno 3 livelli** (almeno 1 hidden) e con funzioni di attivazione **non lineari**.

L'introduzione di almeno un livello nascosto è utile solo se si utilizzano funzioni di attivazione non lineari nei livelli nascosti. È questa non-linearità che permette alla rete di modellare relazioni complesse e non lineari.

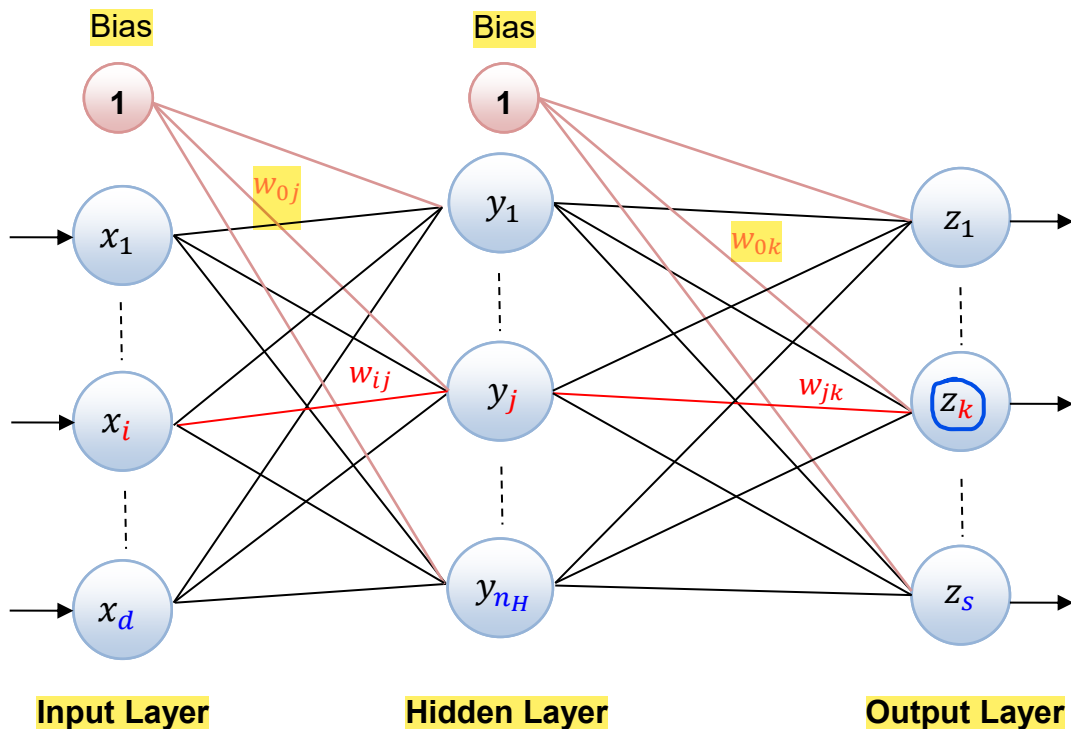
- Un teorema noto come **universal approximation theorem** ^{afferma} ~~asserisce~~ che ogni funzione continua che mappa intervalli di numeri reali su un intervallo di numeri reali può essere approssimata da un MLP con un solo hidden layer.

- questa è una delle motivazioni per cui per molti decenni (fino all'esplosione del deep learning) ci si è soffermati su reti neurali a 3 livelli. D'altro canto, l'esistenza teorica di una soluzione non implica che esista un modo efficace per ottenerla...

Il teorema assicura l'esistenza di una soluzione (cioè, un set di pesi che approssima la funzione), ma non fornisce un metodo efficace per trovarla (cioè, per addestrare la rete in modo rapido e affidabile).

MLP: forward propagation

- Con **forward propagation** (o **inference**) si intende la propagazione delle informazioni in avanti: dal livello di input a quello di output. Una volta addestrata, una rete neurale può semplicemente processare pattern attraverso **forward propagation**.



- nell'esempio una rete a 3 livelli ($d:n_H:s$): input d neuroni, livello nascosto (hidden) n_H neuroni, output s neuroni.

Il numero totale di parametri (pesi e bias) in una rete è ciò che ne definisce la complessità e la capacità di apprendimento.

- Il numero totale di **pesi** (o **parametri**) è: $d \times n_H + n_H \times s + n_H + s$ dove gli ultimi due termini corrispondono ai pesi dei bias.

L'output z_k dipende dall'output del livello precedente, che a sua volta dipende dall'output del livello ancora precedente, e così via

- Il k -esimo valore di output può essere calcolato come: (Eq. 1)

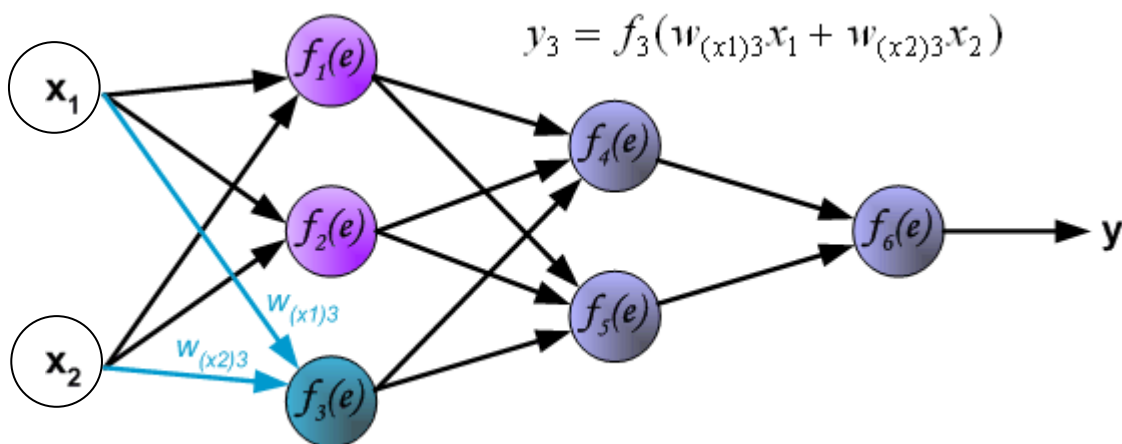
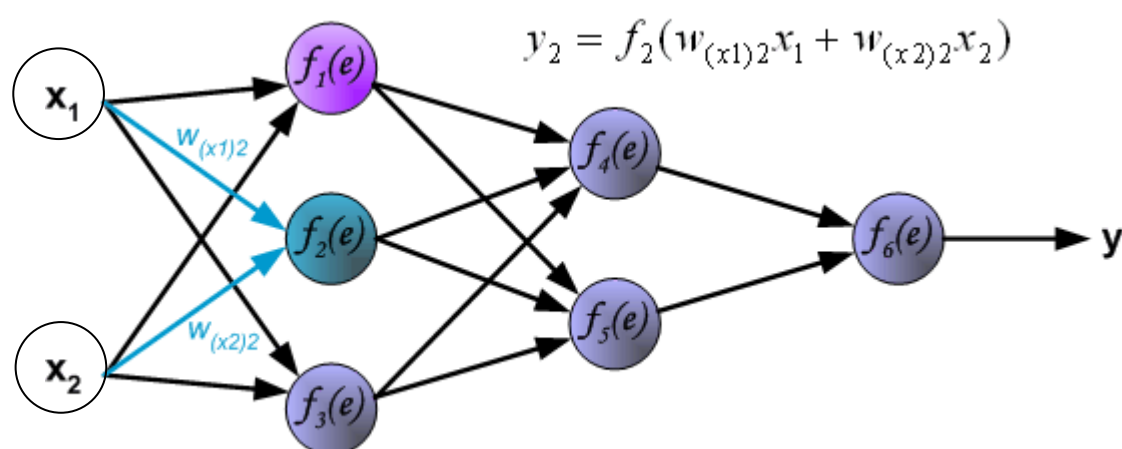
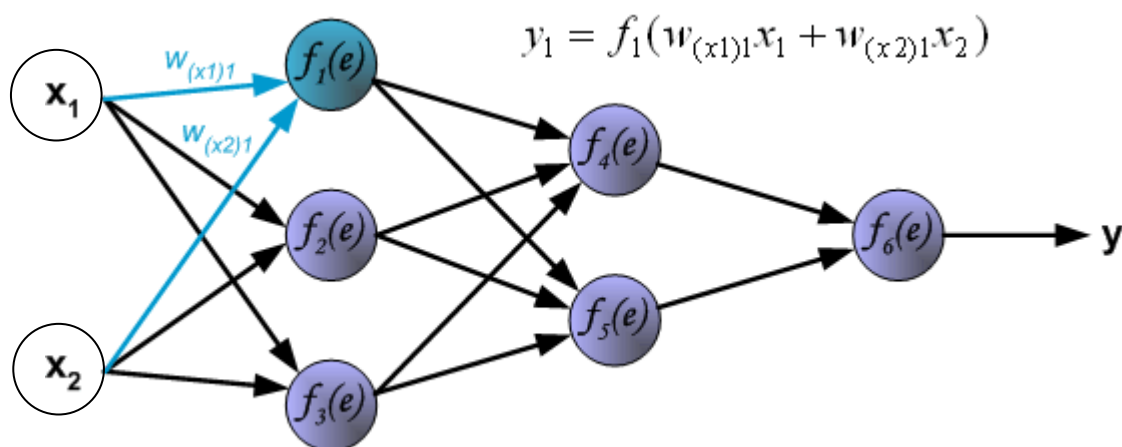
$$z_k = f \left(\sum_{j=1 \dots n_H} w_{jk} \cdot y_j + w_{0k} \right) = f \left(\sum_{j=1 \dots n_H} w_{jk} \cdot f \left(\sum_{i=1 \dots d} w_{ij} \cdot x_i + w_{0j} \right) + w_{0k} \right)$$

↓
 neurone nascosto

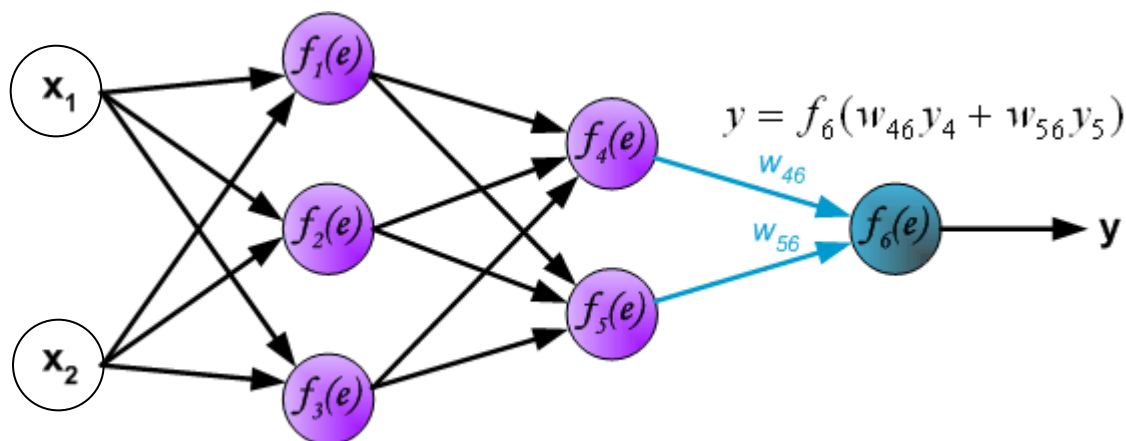
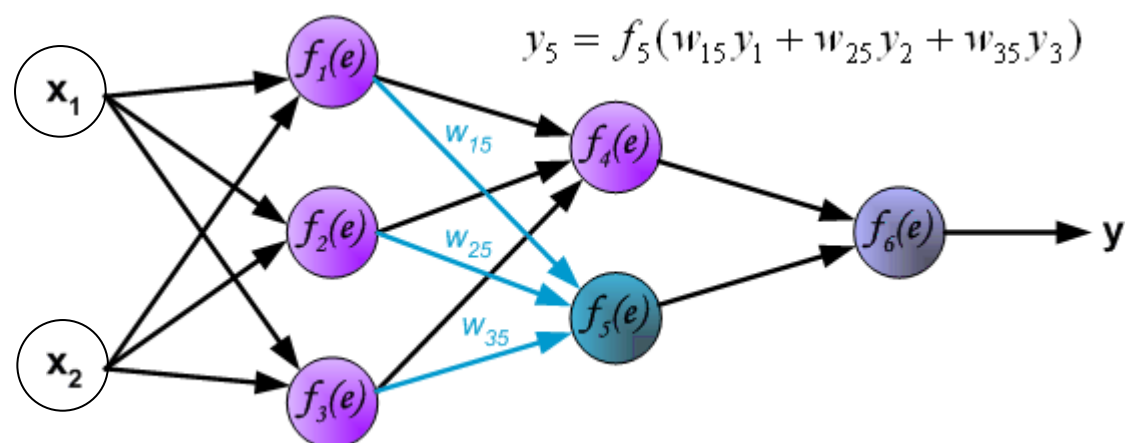
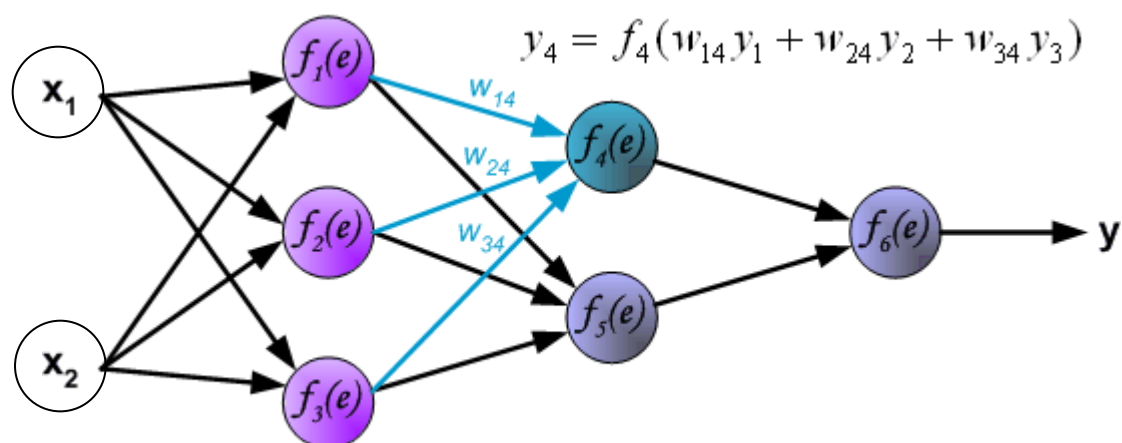
input

Forward propagation: esempio grafico

- Nell'esempio una rete a 4 livelli 2:3:2:1 (2 livelli nascosti); non sono usati bias; nella grafica dell'esempio la notazione è un po' diversa dalla precedente ma facilmente comprensibile.



...continua



MLP: Training

→ L'addestramento consiste nel trovare il set ottimale di pesi (w) e bias (b).

- Fissata la topologia (numero di livelli e neuroni), l'addestramento di una rete neurale consiste nel determinare il valore dei pesi w che determinano il **mapping desiderato** tra input e output.

tipo di

- Che cosa intendiamo per **mapping desiderato**? Dipende dal problema che vogliamo risolvere:

- se siamo interessati ad addestrare una rete neurale che operi come **classificatore**, l'output desiderato è l'etichetta corretta della classe del pattern in input.
- se la rete neurale deve risolvere un problema di **regressione**, l'output desiderato è il valore corretto della variabile dipendente, in corrispondenza del valore della variabile indipendente fornita in input.

- Sebbene i primi neuroni artificiali risalgano agli anni 40', fino a metà degli anni 80' non erano disponibili algoritmi di training efficaci.

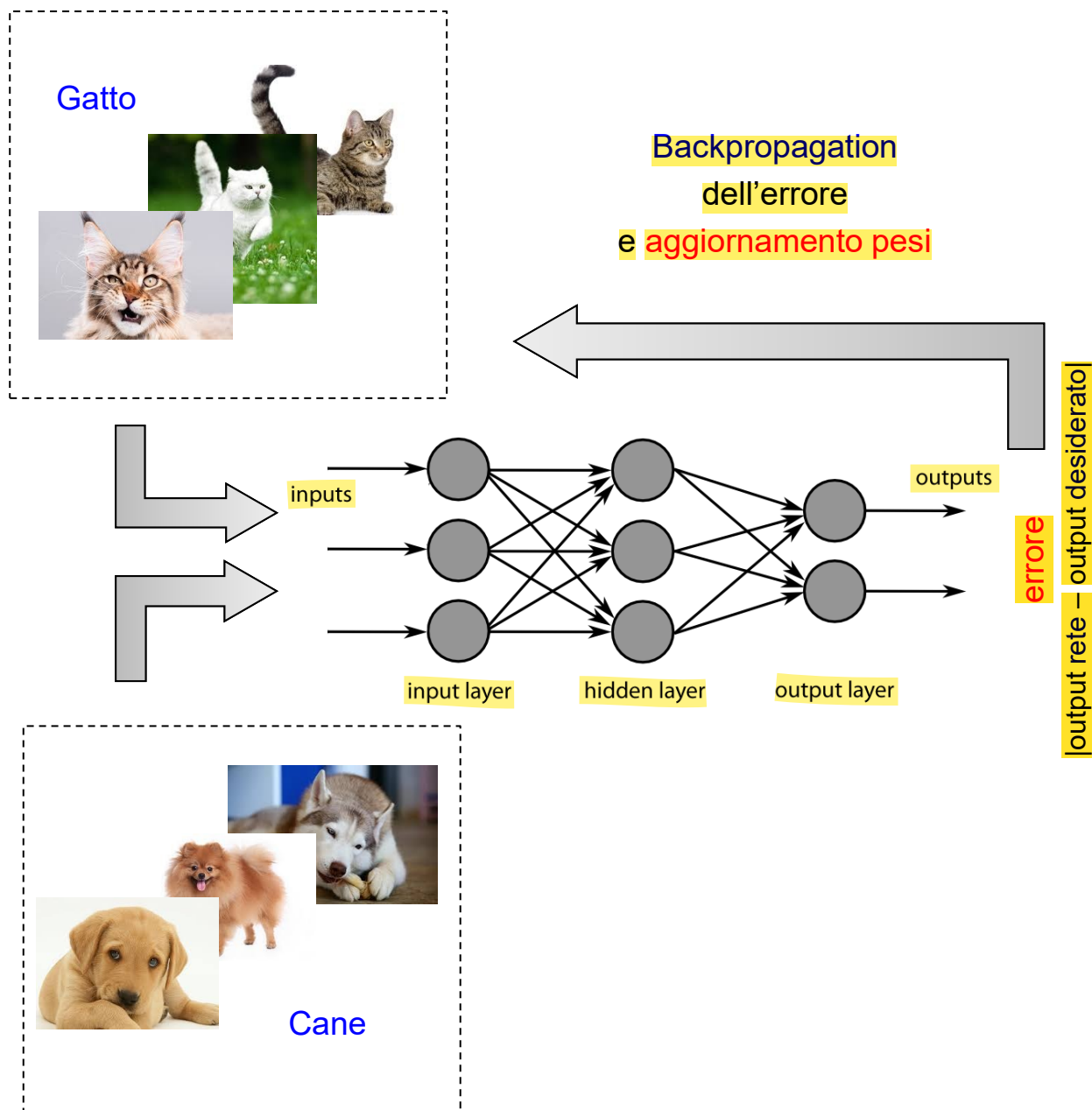
→ L'idea chiave: l'errore commesso in uscita può essere "propagato all'indietro" attraverso la rete, distribuendo la responsabilità dell'errore a ciascun peso in modo quantitativo.

- Nel 1986 Rumelhart, Hinton & Williams hanno introdotto l'algoritmo di **Error Backpropagation** (o semplicemente Backpropagation) suscitando grande attenzione nella comunità scientifica.

Il Credit Assignment è il problema di capire esattamente quale peso, tra i tantissimi presenti, deve essere modificato e di quanto per ridurre l'errore. Il Backpropagation risolve questo problema usando la "regola della catena" (derivata composta): partendo dall'errore finale, torna indietro e assegna a ogni peso la sua precisa quota di responsabilità.

- L'algoritmo risolve il cosiddetto problema del **credit assignment** (ovvero quali pesi sono responsabili per gli errori) attraverso un «**tug of war**» (tiro alla fune) dei training pattern che cercano di spostare il comportamento della rete per minimizzare l'errore su di essi.
- Matematicamente, si tratta dell'applicazione della **regola di derivazione a catena**, idea che però per decenni nessuno aveva applicato con successo in questo contesto.

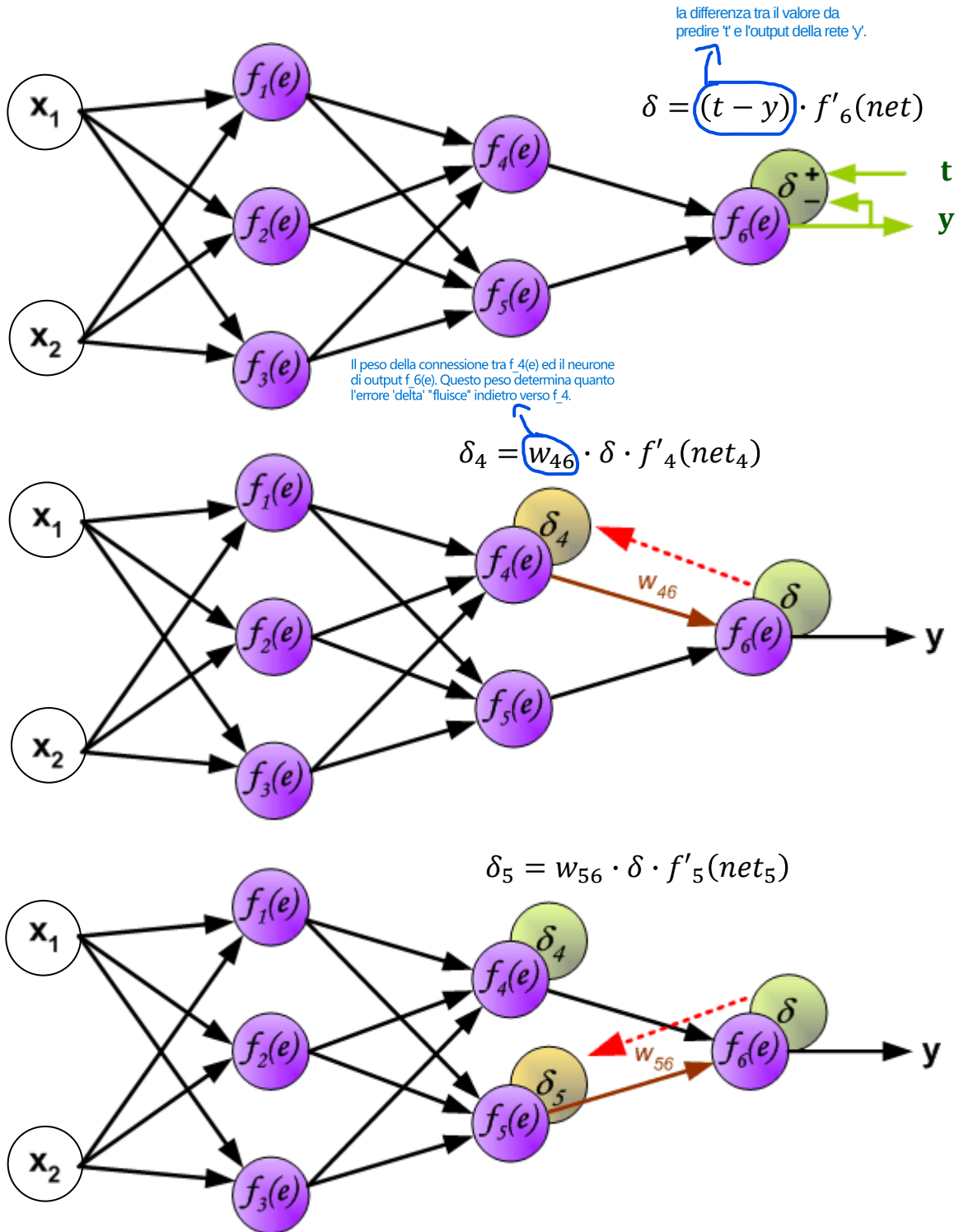
Training (intuizione)



Backpropagation: esempio grafico

Errore sul Livello di Output: Il calcolo inizia dal neurone di output ($f_6(e)$), dove l'errore è noto, perché possiamo confrontare l'output calcolato (y) con l'output desiderato (t , o target).

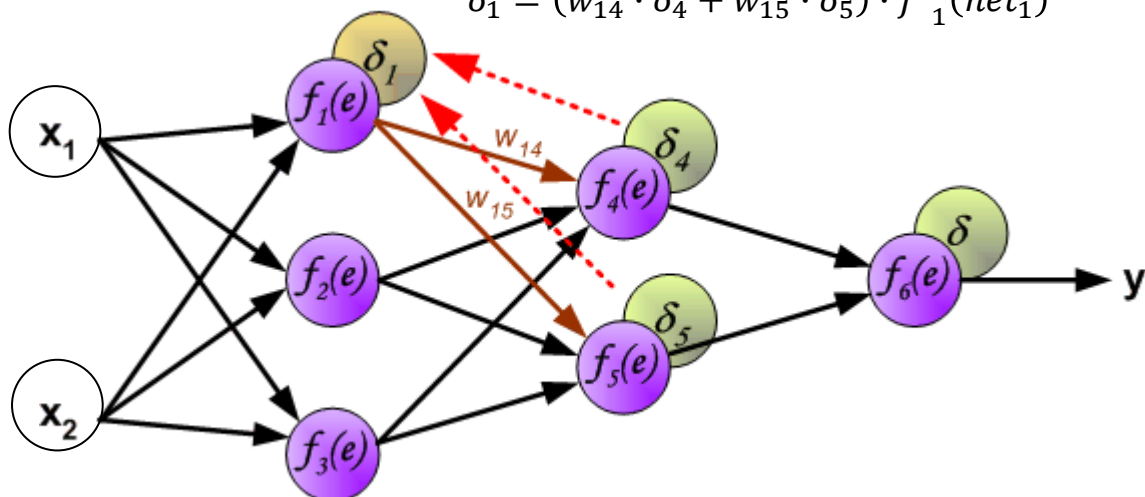
Stessa rete a 4 livelli su cui abbiamo eseguito forward propagation:



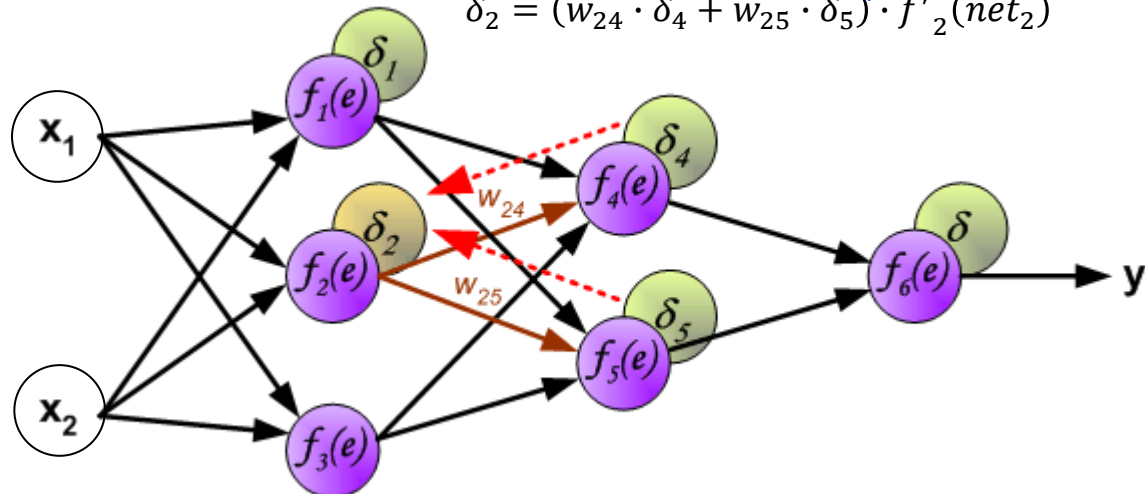
...continua

Si sommano gli errori delta_4 e delta_5, pesati per i rispettivi pesi di connessione. Il risultato viene moltiplicato per la derivata locale $f'_1(net_1)$.

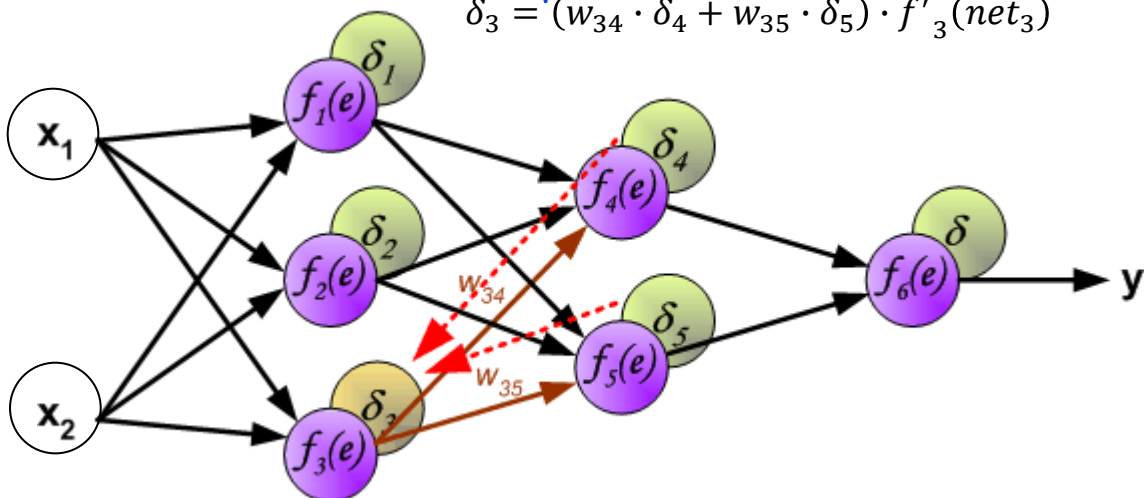
$$\delta_1 = (w_{14} \cdot \delta_4 + w_{15} \cdot \delta_5) \cdot f'_1(net_1)$$



$$\delta_2 = (w_{24} \cdot \delta_4 + w_{25} \cdot \delta_5) \cdot f'_2(net_2)$$

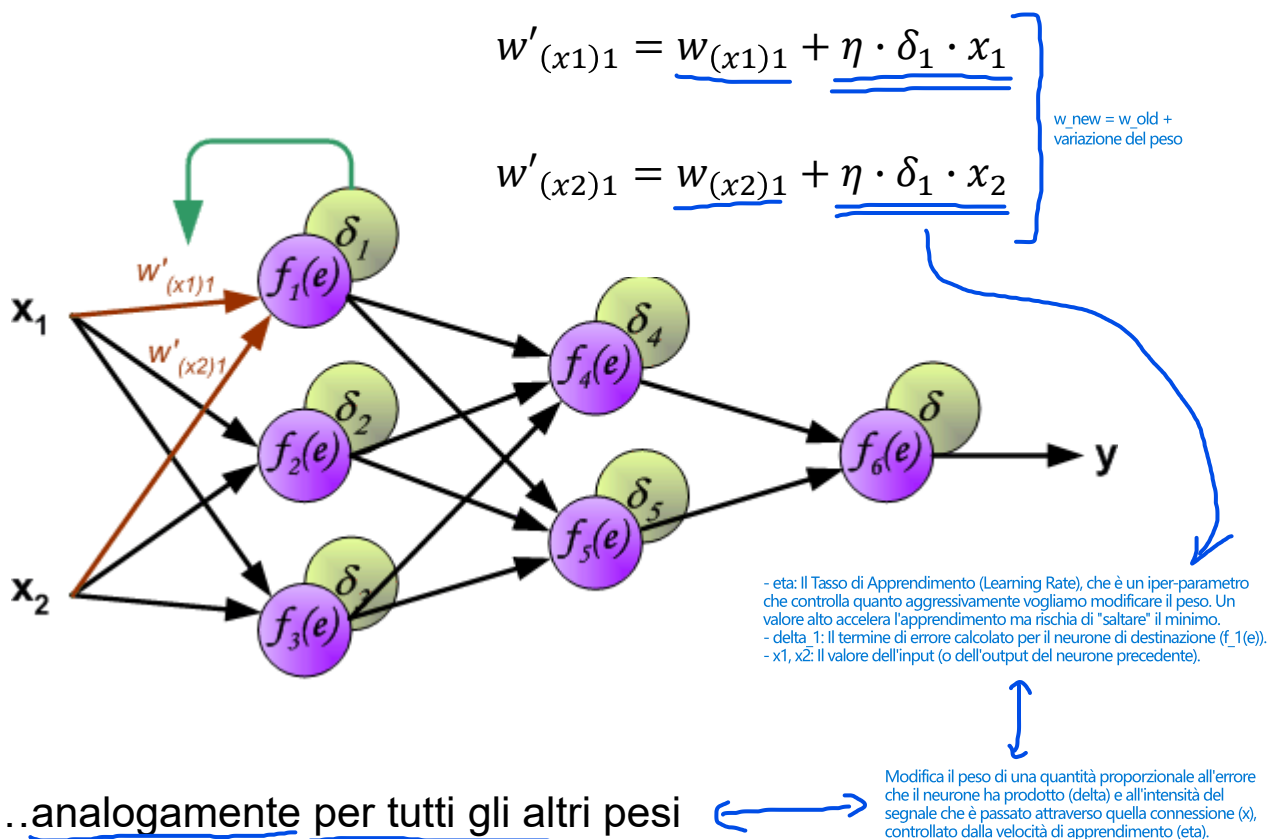


$$\delta_3 = (w_{34} \cdot \delta_4 + w_{35} \cdot \delta_5) \cdot f'_3(net_3)$$



e infine: aggiornamento pesi

Una volta calcolato il termine di errore locale 'delta' per un neurone, si può finalmente aggiornare ogni peso w collegato a quel neurone.



... analogamente per tutti gli altri pesi

Nel seguito proviamo a essere più formali e deriviamo le equazioni per l'aggiornamento dei pesi a partire dalla loss.

Molti concetti introdotti in queste slide possono essere sperimentati e «visivamente» compresi utilizzando il **simulatore di NN** disponibile al link:
<http://playground.tensorflow.org>

Backpropagation: matematicamente

Focalizziamo l'attenzione su un problema di classificazione supervisionata.

- Per gestire un problema di classificazione con s classi e pattern d -dimensionali, si può utilizzare una rete MLP con $d : n_H : s$ neuroni nei tre livelli. Ovvero tanti neuroni di input quante sono le feature e tanti neuroni di output quante sono le classi. n_H è invece un iperparametro: un valore ragionevole è $n_H = \frac{1}{10} n$. num di pattern
- La pre-normalizzazione dei pattern di input (attraverso **min-max scaling**, **standardization** o **whitening**) favorisce la convergenza durante l'addestramento.
- L'addestramento (supervisionato) avviene presentando alla rete pattern di cui è nota la classe e propagando (**forward propagation**) gli input verso gli output attraverso l'equazione (1).

- Dato un pattern di training di classe g , il vettore di output desiderato $\mathbf{t}$ (usando **Tanh** come funzione di attivazione) dell'ultimo livello assume la forma:

Ricordarsi che nelle etichette di un problema di classificazione sono degli interi (es: 3, cioè classe 3 giusta) e quindi devono essere mappati nel vettore precedente

Le etichette dei nostri pattern si possono trasformare in questi vettori fatti in questo modo:

$$\mathbf{t} = [-1, -1 \dots 1 \dots -1]$$

vettore di dimensione s

posizione g (classe giusta)

Tutti -1 tranne nella posizione giusta

Se avessimo utilizzato una sigmoide (che mappa in $[0;1]$) avremmo avuto un vettore con tutti 0 ed un solo 1 sulla posizione giusta

- La differenza tra l'output prodotto della rete e quello desiderato è l'errore della rete. Obiettivo dell'algoritmo di apprendimento è di modificare i pesi della rete in modo da minimizzare l'errore medio sui pattern del training set.
- Prima dell'inizio dell'addestramento, i pesi sono **inizializzati** con valori random in range specifici come meglio descritto in una slide successiva.

Se i pesi vengono inizializzati a random senza un criterio adatto, può portare la rete a non convergere

Backpropagation (1)

- Con riferimento alla rete a 3 livelli della slide 12, siano:
 - $\mathbf{z} = [z_1, z_2 \dots z_s]$ l'output prodotto dalla rete (tramite forward propagation) in corrispondenza del pattern $\mathbf{x} = [x_1, x_2 \dots x_d]$ di classe g fornito in input;
 - $\mathbf{t} = [t_1, t_2 \dots t_s]$ l'output desiderato, dove $t_i = 1$ per $i = g$, $t_i = -1$ altrimenti (vedi slide precedente).
- Scegliendo come loss function la somma dei quadrati degli errori, l'errore per il pattern $\mathbf{x}$ è:

Faccio la differenza dell'output predetto con quello atteso al quadrato e poi sommo i risultati ottenuti per ogni posizione del vettore

$$J(\mathbf{w}, \mathbf{x}) \equiv \frac{1}{2} \sum_{c=1 \dots s} (t_c - z_c)^2$$

ci semplifica i calcoli nelle derivate

output atteso

output generato dalla rete

che quantifica quanto l'output prodotto per il pattern $\mathbf{x}$ si discosta da quello desiderato.

NOTE:

- La dipendenza dai pesi $\mathbf{w}$ è implicita in $\mathbf{z}$. $\mathbf{z}$ è il risultato finale di tutte le somme pesate e attivazioni.
- L'errore $J(\mathbf{w})$ sull'intero training set è la media di $J(\mathbf{w}, \mathbf{x})$ su tutti i pattern $\mathbf{x}$ appartenenti al training set.
- La somma dei quadrati degli errori non è una loss ottimale per la classificazione (spiegato meglio nel seguito) ma per ora preferiamo usare questa loss per semplicità.

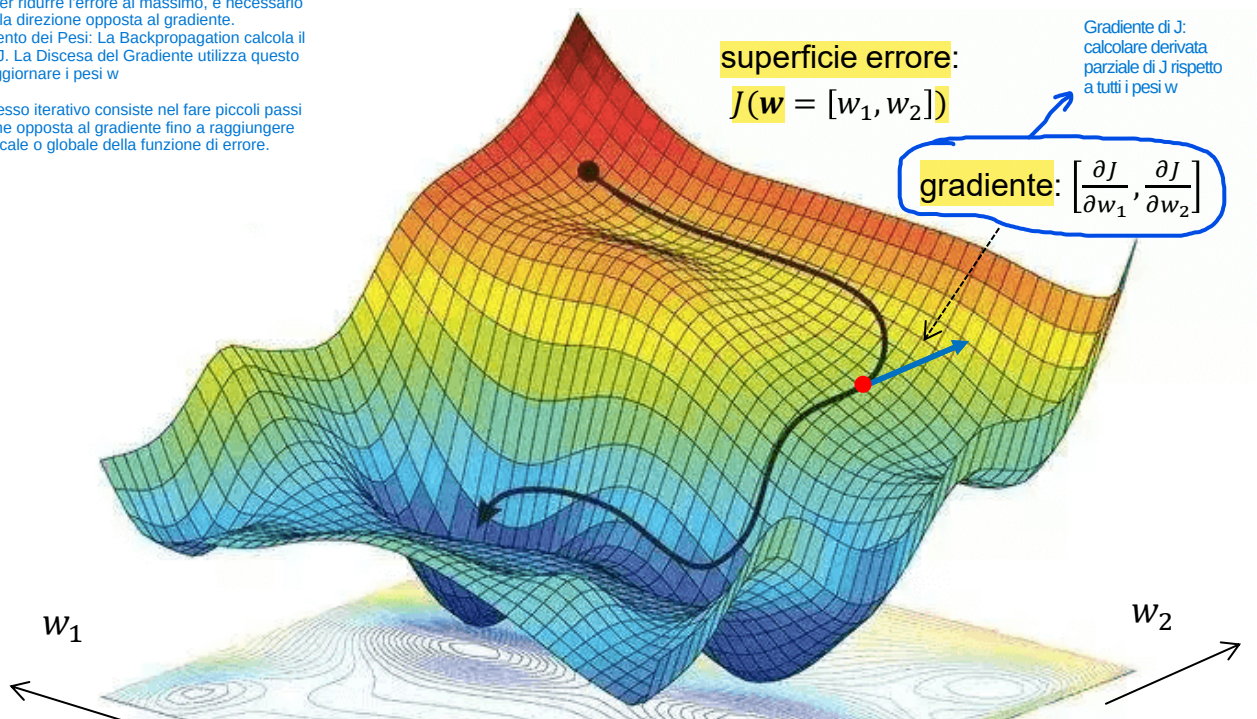
Backpropagation (2)

- $J(\mathbf{w})$ può essere ridotto modificando i pesi $\mathbf{w}$ in direzione **opposta al gradiente di J** . Infatti il (vettore) gradiente indica la direzione di maggior crescita di una funzione (di più variabili) e muovendoci in direzione **opposta** riduciamo (al massimo) l'errore.
- Quando la minimizzazione dell'errore avviene attraverso passi in direzione opposta al gradiente, l'algoritmo backpropagation è denominato anche **gradient descent**. Esistono anche tecniche di minimizzazione del secondo ordine che possono accelerare convergenza (applicabili in pratica solo a reti e training set di piccole/medie dimensioni).

Obiettivo: La minimizzazione dell'errore significa trovare il punto più basso (il minimo) su questa superficie.

- Principio: Per ridurre l'errore al massimo, è necessario muoversi nella direzione opposta al gradiente.
- Aggiornamento dei Pesi: La Backpropagation calcola il gradiente di J . La Discesa del Gradiente utilizza questo valore per aggiornare i pesi $\mathbf{w}$

Questo processo iterativo consiste nel fare piccoli passi nella direzione opposta al gradiente fino a raggiungere un minimo locale o globale della funzione di errore.



superficie errore:
 $J(\mathbf{w} = [w_1, w_2])$

Gradiente di J :
calcolare derivata parziale di J rispetto a tutti i pesi $\mathbf{w}$

gradiente: $\left[\frac{\partial J}{\partial w_1}, \frac{\partial J}{\partial w_2} \right]$

La regola di derivazione a catena è una regola che permette di calcolare la derivata di una funzione composta.

- Nel seguito, consideriamo:

- 1 ■ prima la modifica dei pesi w_{jk} hidden-output. Il calcolo è più semplice perché l'errore è noto al livello di output.
- 2 ■ poi quella dei pesi w_{ij} input-hidden. Questo richiede l'applicazione della regola della catena per propagare l'errore all'indietro.

L'obiettivo è calcolare la derivata parziale della funzione di perdita J rispetto a un generico peso w_{jk} che connette un neurone nascosto j a un neurone di output k.

xi: input
yj: neuroni del livello hidden
zk: neuroni del livello di output

1

Modifica pesi hidden-output

$(1/2) \cdot 2 \cdot (t_k - z_k) \cdot (\text{derivata di } t_k - z_k \text{ dove ho solo la derivata di } -z_k \text{ perché } t_k \text{ è costante})$

Poiché la somma è influenzata solo dal termine relativo al neurone di output k (infatti, solo z_k dipende da w_{jk}), possiamo isolare la derivata:

$$\frac{\partial J}{\partial w_{jk}} = \frac{\partial}{\partial w_{jk}} \left(\frac{1}{2} \sum_{c=1 \dots S} (t_c - z_c)^2 \right) = (t_k - z_k) \frac{\partial (-z_k)}{\partial w_{jk}} =$$

Quindi, la sommatoria sparisce e calcolo solo la derivata di $(t_k - z_k)^2$. Derivata di funzione composta.

solo z_k è influenzato da w_{jk}

cioè mi interessa solo l'output del neurone k e ciò a cui è legato al livello hidden (tutti gli altri output degli altri neuroni sono costanti rispetto al peso w_{jk} e quindi uguali a 0 nel calcolo delle derivate)

Sostituiamo z_k con la sua definizione $f(\text{net}_k)$ (l'output del neurone k è la funzione di attivazione applicata all'input netto net_k)

Applichiamo la Regola della Catena (Chain Rule) su net_k

$$= (t_k - z_k) \frac{\partial (-f(\text{net}_k))}{\partial w_{jk}} = -(t_k - z_k) \cdot \frac{f(\text{net}_k)}{\partial \text{net}_k} \cdot \frac{\partial \text{net}_k}{\partial w_{jk}} =$$

Ricordiamo che net_k è l'input netto al neurone k

La derivata di $f(\text{net}_k)$ rispetto a net_k è semplicemente la derivata della funzione di attivazione: $f'(\text{net}_k)$

In questa sommatoria, c'è un solo valore che dipende da w_{jk} (quello dove $s=j$), tutti gli altri sono costanti rispetto a w_{jk}

$$= -(t_k - z_k) \cdot f'(\text{net}_k) \cdot \frac{\partial \sum_{s=1 \dots n_H} w_{sk} \cdot y_s}{\partial w_{jk}} = -(t_k - z_k) \cdot f'(\text{net}_k) \cdot y_j$$

quindi, mi rimane la derivata di $(w_{jk} \cdot y_j)$ rispetto a w_{jk}

semplifico w_{jk} sia al num. sia al denom. ed ottengo solo y_j

posto $\delta_k = (t_k - z_k) \cdot f'(\text{net}_k)$ (Eq. 2)

È conveniente raggruppare i primi due termini in un unico fattore, chiamato errore locale del neurone di output k (delta_k)

allora $\frac{\partial J}{\partial w_{jk}} = -\delta_k \cdot y_j$ (Eq. 3)

Questo delta_k rappresenta l'errore residuo del neurone k pesato per la pendenza locale (derivata prima) della sua funzione di attivazione.

pertanto il peso w_{jk} può essere aggiornato come:

$$w_{jk} = \overline{w_{jk}} + \eta \cdot \delta_k \cdot y_j \quad (\text{Eq. 4})$$

I pesi li aggiorniamo in direzione opposta al gradiente per quel peso

dove η è il learning rate.

mi dice quanto devo andare veloce nel fare questi aggiornamenti

al posto del meno che si trova nella formula Eq. 3, visto che devo andare in direzione opposta al gradiente, sostituisco con un più

mi dicono come intervenire sul peso



N.B. se si usano i bias il peso w_{0k} si aggiorna ponendo $y_0 = 1$

L'obiettivo è calcolare la derivata parziale della funzione di perdita J rispetto a un peso w_{ij} che connette un input x_i a un neurone nascosto y_j . Questa derivazione è più complessa perché l'errore deve essere propagato attraverso il livello nascosto dal livello di output.

xi: input
yj: neuroni del livello hidden
zk: neuroni del livello di output

2 Modifica pesi input-hidden

Rispetto a prima, in questo passaggio, devo portarmi avanti la sommatoria perché ci sono più z_c che dipendono da w_{ij} (e non solo uno come prima)

$$\frac{\partial J}{\partial w_{ij}} = \frac{\partial}{\partial w_{ij}} \left(\frac{1}{2} \sum_{c=1 \dots s} (t_c - z_c)^2 \right) = - \sum_{c=1 \dots s} (t_c - z_c) \frac{\partial z_c}{\partial w_{ij}}$$

Poiché il peso w_{ij} influenza l'output z_c di tutti i neuroni finali tramite il neurone nascosto y_j , la derivata deve sommare i contributi di tutti i neuroni di output s

$$= - \sum_{c=1 \dots s} (t_c - z_c) \frac{\partial z_c}{\partial net_c} \cdot \frac{\partial net_c}{\partial w_{ij}}$$

Chain Rule

$$= - \sum_{c=1 \dots s} (t_c - z_c) \cdot f'(net_c) \cdot \frac{\partial net_c}{\partial w_{ij}}$$

δ_c vedi Eq. 2

$$\frac{\partial net_c}{\partial w_{ij}} = \frac{\partial}{\partial w_{ij}} \sum_{r=1 \dots n_H} w_{rc} \cdot y_r = \frac{\partial}{\partial w_{ij}} \sum_{r=1 \dots n_H} w_{rc} \cdot f(net_r)$$

solo net_j è influenzato da w_{ij}

$$= \frac{\partial}{\partial w_{ij}} (w_{jc} \cdot f(net_j)) = w_{jc} \frac{\partial f(net_j)}{\partial net_j} \cdot \frac{\partial net_j}{\partial w_{ij}}$$

$$= w_{jc} \cdot f'(net_j) \cdot \frac{\partial}{\partial w_{ij}} \sum_{q=1 \dots d} w_{qj} \cdot x_q = w_{jc} \cdot f'(net_j) \cdot x_i$$

la somma pesata degli errori delta_c non è l'errore di output in sé, ma la quantità di errore d'uscita che viene reindirizzata a ritroso verso il neurone nascosto j, fornendogli il "credito" o la "colpa" per il suo contributo all'errore finale.

$$\frac{\partial J}{\partial w_{ij}} = - \sum_{c=1 \dots s} \delta_c \cdot w_{jc} \cdot f'(net_j) \cdot x_i = -x_i \cdot f'(net_j) \sum_{c=1 \dots s} w_{jc} \cdot \delta_c$$

Somma degli errori a valle:
- Cosa sono i delta_c? Sono i termini di errore calcolati per ogni neurone di output c. Questi delta_c sono gli unici che dipendono direttamente dal valore target desiderato (t) e sono calcolati per primi: $\delta_c = (t_c - z_c) \cdot f'(net_c)$.
- Perché sommare: Un neurone nascosto j non è collegato a un solo neurone di output, ma a tutti i neuroni di output $c=1, \dots, s$. Per sapere quanto il neurone j è responsabile dell'errore totale, dobbiamo sommare l'errore ricevuto da tutte le connessioni a valle.

posto $\delta_j = f'(net_j) \cdot \sum_{c=1 \dots s} w_{jc} \cdot \delta_c$ (Eq. 5)

allora $\frac{\partial J}{\partial w_{ij}} = -\delta_j \cdot x_i$ (Eq. 6) La formula finale è sempre la stessa (l'unica cosa è che cambia il delta)

pertanto il peso w_{ij} può essere aggiornato come:

$$w_{ij} = w_{ij} + \eta \cdot \delta_j \cdot x_i$$

(Eq. 7)

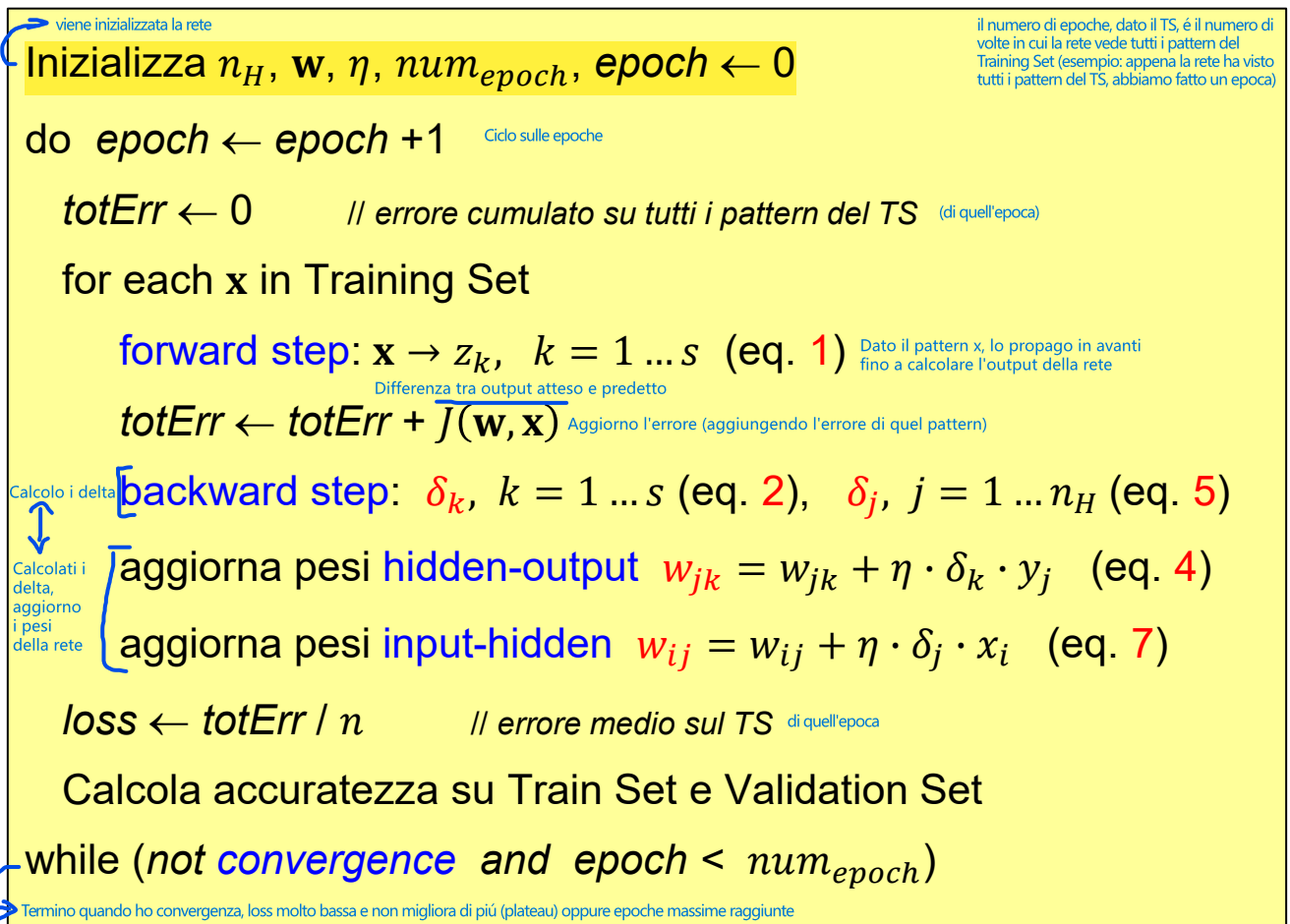
Mi dicono come intervenire sul peso

Ogni errore delta_c viene moltiplicato per il peso w_{jc} della connessione che lo lega al neurone nascosto j. Questo fattore di peso funge da moltiplicatore di responsabilità. Se il peso w_{jc} è grande, significa che il neurone j ha esercitato una forte influenza sull'output z_c . Di conseguenza, è giusto che il neurone j riceva un segnale di errore (il delta_c) più grande da quella connessione, perché era più "responsabile" di quel risultato.

N.B. se si usano i bias il peso w_{0j} si aggiorna ponendo $x_0 = 1$

Algoritmo Backpropagation (Online)

Nella versione **online**, i pattern del training set sono presentati sequenzialmente e i pesi sono aggiornati dopo la presentazione di ogni pattern:



- La convergenza si valuta monitorando l'andamento del loss e l'accuratezza sul validation ^{e training} set (vedi slide *Fondamenti*).
- L'approccio on-line richiede l'aggiornamento dei pesi a seguito della presentazione di ogni pattern. Problemi di efficienza (molti update di pesi) e robustezza (in caso di outliers passi in direzioni sbagliate).

- Per rendere efficiente il calcolo dell'output della rete, parto dall'inizio e vado avanti da sinistra verso destra.
- Per rendere efficiente il calcolo dei delta della rete, parto dalla fine e vado indietro da destra verso sinistra (in modo tale da riutilizzare i delta dei passi precedenti).

Stochastic Gradient Descent (SGD)

- È l'approccio più utilizzato per implementare backpropagation:
 - a ogni epoca, gli n pattern del training set sono ordinati in modo casuale e poi suddivisi, considerandoli sequenzialmente, in gruppi (denominati **mini-batch**) di uguale dimensione $size_{mb}$.
 - se $size_{mb} = 1 \rightarrow$ «stochastic» **online** Online SGD
 - se $size_{mb} = n \rightarrow$ «full» **batch** (un solo mini-batch grande quanto il TS, si chiama Full-Batch SGD)
 - i valori del **gradiente** sono (algebricamente) **accumulati** in variabili temporanee (una per ciascun peso); l'aggiornamento dei pesi avviene solo quando tutti i pattern di un mini-batch sono stati processati.

gli intermedi
usano Mini-
Batch

Inizializza n_H , w , η , num_{epoch} , $size_{mb}$, $epoch \leftarrow 0$

do $epoch \leftarrow epoch + 1$

random sort the Training Set (n patterns)

Meglio se ogni batch è equamente rappresentato (cioè eterogeneo e bilanciato tra le classi)

Mescolare in modo randomico il training set (per evitare che nello stesso batch finiscano i pattern di una stessa classe)

Va bene anche farlo una sola volta anziché ad ogni epoca

for each **mini-batch** B of $size_{mb}$

Questo ad ogni epoca

reset gradiente

Esempio:
TS: 1000 pattern
size batch: 20 pattern
questo ciclo ha 50 iterazione (1 epoca)

for each x in B Ciclo sui pattern del mini-batch

forward step

backward step Gradiente viene accumulato // *gradiente cumulato su tutti i pattern del mini-batch*

Gradiente viene sommato con quello del pattern precedente dello stesso batch

aggiorna pesi w con learning rate η Finiti i pattern del mini-batch, aggiorni i pesi (una volta sola per mini-batch)

Calcola Loss

Calcola accuratezza su Training Set e Validation Set

while (not **convergence** and $epoch < num_{epoch}$)

Terminologia (SGD)

■ Attenzione alla terminologia:

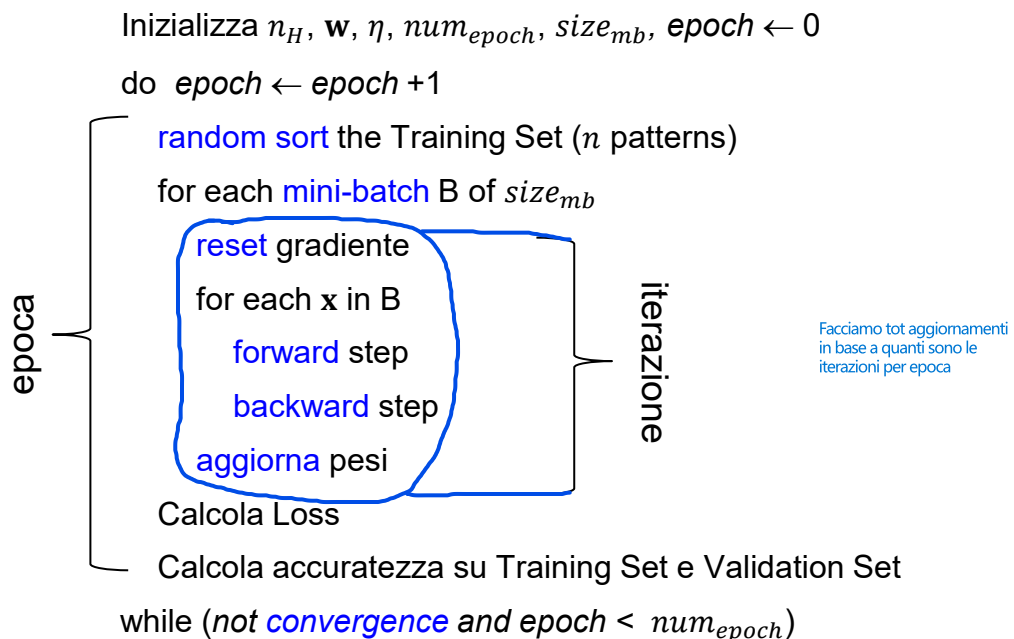
1 epoca = viene visto tutto il TS dalla rete

■ Per **epoca** (epoch) si intende la presentazione (1 volta) alla rete di tutti i pattern del training set.

1 iterazione = viene visto 1 mini-batch dalla rete

■ Per **iterazione** (iteration) si intende la presentazione (1 volta) dei pattern costituenti un mini-batch e il conseguente aggiornamento dei pesi. (numero di iterazione 1:1 con numero di mini-batch/aggiornamenti dei pesi)

- $n/size_{mb}$ è il numero di iterazioni per epoca. Se non è un valore intero all'ultima iterazione si processano meno pattern.



Invece di num_{epoch} e $size_{mb}$ alcuni tool (tra cui Caffe) richiedono in input il numero totale di iterazioni num_{iter} e $size_{mb}$; in questo caso il numero di epoche (non necessariamente intero) è:

Stai calcolando il numero totale di esempi processati dalla rete durante tutto l'addestramento.

Ora, per sapere quante "epoche" hai fatto, devi dividere questo numero totale per la grandezza del tuo dataset.

$$num_{epoch} = \frac{num_{iter} \cdot size_{mb}}{n}$$

L'uso di mini-batch piccoli introduce rumore nel gradiente, rendendo instabili i minimi sharp e favorendo la convergenza verso minimi flat, che risultano più robusti alle variazioni dei dati e quindi migliori in termini di generalizzazione.
- flat minima: Sono più stabili, poiché piccole variazioni nei pesi (dovute a nuovi dati) causano solo un piccolo aumento dell'errore.
- sharp minima: Sono meno stabili; piccole variazioni dei pesi causano un grande aumento dell'errore.

Tarare il minibatch size

Definire il valore ottimale di $size_{mb}$ (come per altri iperparametri) non è banale.

■ Se consideriamo la sola **efficienza**:

■ $size_{mb}$ maggiori aumentano velocità del training (meno aggiornamenti del gradiente e processing più efficiente di minibatch grandi).
(uso più efficiente di GPU/CPU (parallelismo))

■ Attenzione: $size_{mb}$, in modelli di grandi dimensioni, è limitato dalla memoria disponibile nella GPU.

■ Relativamente alla **generalizzazione** però:

■ Si è osservato che i modelli generalizzano maggiormente quando convergono a flat minima (non sharp).
minima

perché il gradiente di un piccolo batch è più rumoroso rispetto ad un mini-batch grande.

■ Minibatch piccoli o di medie dimensioni sono più esplorativi (randomness) e meno attratti da sharp minima (da cui, con large minibatch, è difficile uscire).
mini-batch
Questa randomness aiuta la Discesa del Gradiente a sfuggire dai sharp minima, verso i quali i large mini-batch sono più attratti e da cui è difficile uscire.

■ $size_{mb}$ e il learning rate η non sono indipendenti:

■ In genere, con minibatch più grandi si può usare un learning rate più aggressivo (maggiore) grazie a un effetto «media» più forte nelle correzioni.
Questo perché l'accumulo del gradiente su un batch più grande fornisce una stima media più stabile e affidabile del gradiente totale, riducendo il "rumore" che altrimenti richiederebbe un passo (eta) più piccolo per evitare di saltare il minimo.

■ Alcuni autori propongono **aumento lineare** di η all'aumentare di $size_{mb}$
In pratica, $size_{mb}$ viene spesso scelto in potenze di 2 (es. 32, 64, 128, 256) per ottimizzare l'uso della memoria della GPU.

SoftMax: Output Probabilistico

- Se nel livello di output si utilizza come funzione di **attivazione**:
 - **Tanh**: i valori di uscita sono nell'intervallo $[-1...1]$, e quindi non sono probabilità. Inoltre, la somma delle probabilità di ogni classe non è 1
 - **Sigmoid** (standard logistic function): i valori di uscita sono compresi nell'intervallo $[0...1]$, ma non abbiamo nessuna garanzia che la somma sui neuroni di uscita sia 1 (requisito fondamentale affinché si possano interpretare come distribuzione probabilistica).

$$\sum_{c=1...S} z_c \neq 1$$

Nei problemi di classificazione, l'obiettivo non è solo dare un punteggio ad ogni classe, ma interpretare tali punteggi come la probabilità che il pattern di input appartenga a quella classe.

Quando una rete neurale è utilizzata come **classificatore multi-classe**, l'impiego della funzione di attivazione **softmax** consente di trasformare i valori di **net** prodotti dall'ultimo livello della rete in probabilità delle classi:

- Il livello di attivazione net_k dei singoli neuroni dell'ultimo livello si calcola nel modo consueto, ma come **funzione di attivazione** per il neurone k -esimo si utilizza:

$$z_k = f(net_k) = \frac{e^{net_k}}{\sum_{c=1...S} e^{net_c}}$$

Il valore dell'esponenziale di net_k viene normalizzato per la somma di tutti gli esponenziali di z dell'ultimo livello della rete, così da ottenere una probabilità

- i valori z_k prodotti possono **essere interpretati come probabilità delle classi**: appartengono a $[0...1]$ e la loro somma è 1.
- l'esponenziale (utile nella combinazione con Cross Entropy Loss, vedi slide seguenti) interpreta i valori **net** come **unnormalized log probabilities** delle classi.

Cross-Entropy Loss

- Nelle slide precedenti abbiamo utilizzato per un problema di **classificazione multi-classe** la funzione di attivazione **Tanh** e la loss function **Sum of Squared Error**.

- questa scelta **non è ottimale**, in quanto i valori di output non rappresentano probabilità, e la **non imposizione** del vincolo di somma a 1, rende (in genere) meno efficace l'apprendimento.

- Per un problema di **classificazione multi-classe**, si consiglia l'utilizzo di **Cross-Entropy** (detta anche **Multinomial Log Loss** o **Multinomial Logistic Regression Loss** o) come loss function:

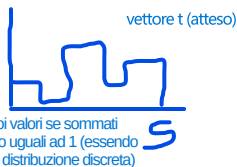
La cross-entropy mi dice quanto sono simili/diversi tra di loro le distribuzioni discrete p e q

- La cross-entropy tra due distribuzioni discrete p e q (che fissata p misura quanto q differisce da p) è definita da:

$$H(p, q) = - \sum_v p(v) \cdot \log(q(v))$$

probabilità vera della classe v probabilità predetta dalla rete per la classe v

v indica una classe (sommatoria su tutte le classi possibili)



- Nell'impiego come loss function: p (fissato) è il vettore target, mentre q il vettore di output della rete.

Il valore della cross-entropy aumenta, all'aumentare dell'errore tra i valori di q e p (differiscono)

- Il valore minimo di H (sempre ≥ 0) si ha quando il vettore target coincide con l'output. **Attenzione** nella formula il logaritmo non esiste in 0, ma gli output forniti da softmax e sigmoid non valgono mai 0 (anche se vi possono tendere asintoticamente).

Un neurone può predire 0 e quindi creare problemi nel calcolo di $\log(0)$?
Un neurone al massimo può predire un valore vicinissimo a 0, mai 0 preciso

Questo perché se net_k della softmax è 0, quindi al numeratore = 1, per avere 0 come risultato si deve avere al denominatore l'infinito

- Per approfondimenti e significato in teoria dell'Informazione:
<https://rdipietro.github.io/friendly-intro-to-cross-entropy-loss/>

Cross-Entropy e One-hot targets

Quando si utilizza la Cross-Entropy Loss e il vettore target $\mathbf{t}$ è in formato One-Hot (tutti 0 tranne un 1 nella posizione g della classe corretta), la formula generale della Cross-Entropy si semplifica drasticamente.

Quando il target vector per $\mathbf{x}$ assume la forma **one-hot** (tutti 0 tranne un 1 per la classe corretta g):

$$\mathbf{t} = [0, 0 \dots \underset{\substack{\swarrow \text{posizione } g}}{1} \dots 0]$$

allora:

$$J(\mathbf{w}, \mathbf{x}) \equiv H(\mathbf{t}, \mathbf{z}) = -\log(z_g)$$

Poiché nel vettore target $\mathbf{t}$ solo il termine corrispondente alla classe corretta g è 1, e tutti gli altri sono 0, la somma si riduce ad un solo termine

Se z_g è ottenuta con funzione di attivazione **softmax**, allora:

$$J(\mathbf{w}, \mathbf{x}) = -\log\left(\frac{e^{net_g}}{\sum_{c=1 \dots s} e^{net_c}}\right) = -net_g + \log \sum_{c=1 \dots s} e^{net_c}$$

abbiamo stesso risultato utilizzando cross-entropy come loss e softmax nell'ultimo livello rispetto a quello visto prima

Per la derivazione del gradiente (backpropagation) si veda l'ottimo report "[Notes on Backpropagation](#)" di Peter Sadowski

Loss Function per Classificazione

 softmax + cross-entropy

■ **Classificazione multi-classe**: utilizzare funzione di attivazione softmax nel livello finale e cross-entropy come loss function.

- se i target vector t assumono la forma one-hot (default) si utilizza la semplificazione della slide precedente.
- in caso di incertezza sulla classe corretta, è possibile utilizzare «soft target», con singoli valori tra 0 e 1 e somma a 1. In questo si usa formula completa di cross-entropy.

 1 neurone + sigmoid + binary cross-entropy

■ **Classificazione binaria**: è possibile ricadere nel caso precedente e trattare il problema come multiclasse con 2 classi, ma risulta preferibile (spesso convergenza migliore):

- utilizzare 1 neurone di uscita che codifica la probabilità della sola prima classe (la seconda probabilità è il complemento a 1). A tal fine si utilizza la funzione di attivazione sigmoid che produce valori in $[0...1]$ per il primo neurone.
- la normalizzazione a 1 della probabilità delle due classi è implicita nella semplificazione di cross-entropy adottata (multinomial log loss). Si noti che con 1 neurone t, z sono valori scalari in $[0...1]$:

$$H(t, z) = -(t \log(z) + (1 - t) \log(1 - z))$$

 sigmoid su ogni uscita + somma di binary cross-entropy

■ Un caso particolare è la cosiddetta **classificazione multi-label** dove un pattern **può appartenere contemporaneamente a più classi**. Ad esempio un'immagine che contiene sia un cane che un gatto potrebbe essere classificata come appartenente a 2 classi.

- in questo caso ogni uscita è in $[0...1]$ ma deve essere rilassato il vincolo di somma a 1 per gli output. Si può ottenere come estensione della classificazione binaria (sopra) utilizzando attivazioni sigmoid e sommando $H(t, z)$ su più neuroni.

 L'output z di un neurone non deve influenzare gli altri.

Loss Function per Regressione

- Consideriamo un problema di regressione dove sia la variabile indipendente (input) che quella dipendente (output) sono vettori.
- Utilizzando la terminologia introdotta per la regressione stiamo affrontando un problema di **multivariate multiple regression**
- La rete neurale è in grado di trovare un mapping non lineare tra input e output, pertanto il termine **linear** non si applica perché la rete introduce non linearità nei layer nascosti.
- Se l'output assume valori **reali non limitati**, si consiglia di non utilizzare funzione di attivazione nel livello finale: i target vector $\mathbf{t}$ non sono soggetti a vincoli:

$$\mathbf{t} = [t_1, t_2 \dots t_s]$$

In ogni caso è consigliabile, per maggiore stabilità, riscalarlo l'output nel range $[0, 1]$.

Prima di addestrare la rete, si normalizzano (o si standardizzano) i valori di input e output del training set. La rete quindi impara a prevedere valori normalizzati, più gestibili numericamente e con gradienti stabili. Quando si valuta la rete sul test set o si fanno predizioni finali, si applica la trasformazione inversa (de-normalizzazione) per riportare i valori alla scala originale.

- Come **loss function** possiamo usare una qualsiasi di quelle introdotte in precedenza: **MSE, MAE, MAPE**.

Inizializzazione dei pesi

- L'inizializzazione random dei pesi in un range ottimale è molto importante per la convergenza della rete.
- La «vecchia» inizializzazione con distribuzione normale a media 0 e deviazione standard 1, tende a produrre una varianza degli output di un livello maggiore di quella degli input. Ciò combinato con funzioni di attivazione saturanti (come sigmoid e tanh) può portare a correzioni quasi nulle del gradiente e quindi la rete a non convergere.

Questa tecnica è progettata per funzionare bene con funzioni di attivazione simmetriche come tanh

- **Inizializzazione Xavier** (o Glorot): Xavier Glorot e Yoshua Bengio hanno dimostrato nel 2010 che per mantenere la varianza di output simile a quella di input si possono inizializzare i pesi:

- Con distribuzione normale a media 0 e deviazione standard:

$$\sigma = \sqrt{\frac{1}{fan_{avg}}} \quad \text{dove } fan_{avg} \text{ è la media tra il numero di input}$$

(fan_{in}) e output (fan_{out}) del livello. $(fan_{in}+fan_{out}/2)$

numero di neuroni
nello strato precedente

numero di neuroni
nello strato corrente

- Con distribuzione uniforme in: $\left[-\sqrt{\frac{3}{fan_{avg}}}, +\sqrt{\frac{3}{fan_{avg}}} \right]$

- In combinazione con la funzione di attivazione Relu (funzione non saturante introdotta nel seguito), l'inizializzazione He è utilizzata di default, dove:

l'inizializzazione dei pesi avviene con distribuzione
normale a media 0 e deviazione standard

$$\sigma = \sqrt{\frac{2}{fan_{in}}}$$

Regolarizzazione

- Per ridurre il rischio di overfitting del training set da parte di una rete neurale con molti parametri (pesi), si possono utilizzare tecniche di regolarizzazione. La regolarizzazione è molto importante quando il training set non ha grandi dimensioni rispetto alla capacità^{/complessità} del modello.
- reti neurali i cui pesi, o parte di essi, assumono valori piccoli (vicino allo zero) producono output più regolare e stabile, portando spesso a migliore generalizzazione.
- Per spingere la rete ad adottare pesi di valore piccolo si può aggiungere un termine di regolarizzazione alla loss. Ad esempio nel caso di Cross-Entropy Loss: (si cercheranno pesi ottimali e piccoli)

$$J_{Tot} = J_{Cross-Entropy} + J_{Reg}$$

- Nella regolarizzazione L2 il termine aggiunto corrisponde alla somma dei quadrati di tutti i pesi della rete:

$$J_{Reg} = \frac{1}{2} \lambda \sum_i w_i^2$$

Tende a rendere tutti i pesi piccoli, ma raramente li porta esattamente a zero.

- Nella regolarizzazione L1 si utilizza il valore assoluto:

$$J_{Reg} = \lambda \sum_i |w_i|$$

Tende a spingere numerosi pesi esattamente a zero, lasciando solo i pesi più importanti diversi da zero.

- In entrambi i casi il parametro λ regola la forza della regolarizzazione.
- L1 può avere un effetto sparsificante (ovvero portare numerosi pesi a 0) maggiore di L2. Infatti quando i pesi assumono valori vicini allo zero il calcolo del quadrato in L2 ha l'effetto di ridurre eccessivamente i correttivi ai pesi rendendo difficile azzerarli.

L2: correttivo lambda proporzionale a w_i , quindi piccolo per pesi piccoli, allora difficile azzerarli
L1: correttivo lambda costante, allora efficace a rendere molti pesi esattamente 0

Regolarizzazione → Weight Decay

■ Consideriamo la regolarizzazione L2.

- il gradiente della J_{Tot} rispetto ad uno dei parametri della rete corrisponde alla somma del gradiente di $J_{Cross-Entropy}$ e del gradiente di J_{Reg} . Quest'ultimo vale:

$$\frac{\partial J_{Reg}}{\partial w_k} = \frac{\partial}{\partial w_k} \left(\frac{1}{2} \lambda \sum_i w_i^2 \right) = \lambda \cdot w_k$$

Essendo la derivata rispetto ad un k , viene eliminata la sommatoria e rimane un solo elemento (dove $1/2$ si semplifica con 2)

- pertanto l'aggiornamento dei pesi a seguito di backpropagation include un ulteriore termine denominato weight decay (decadimento del peso) che ha l'effetto di tirarlo verso lo 0.
- Ad esempio considerando l'aggiornamento pesi di SGD (vedi precedenti equazioni 4 e 7) :

$$w_k = w_k - \eta \cdot \frac{\partial J_{Cross-Entropy}}{\partial w_k}$$

diventa:

$$w_k = w_k - \eta \left(\frac{\partial J_{Cross-Entropy}}{\partial w_k} + \lambda \cdot w_k \right)$$

parte riguardante la loss function (cross-entropy)

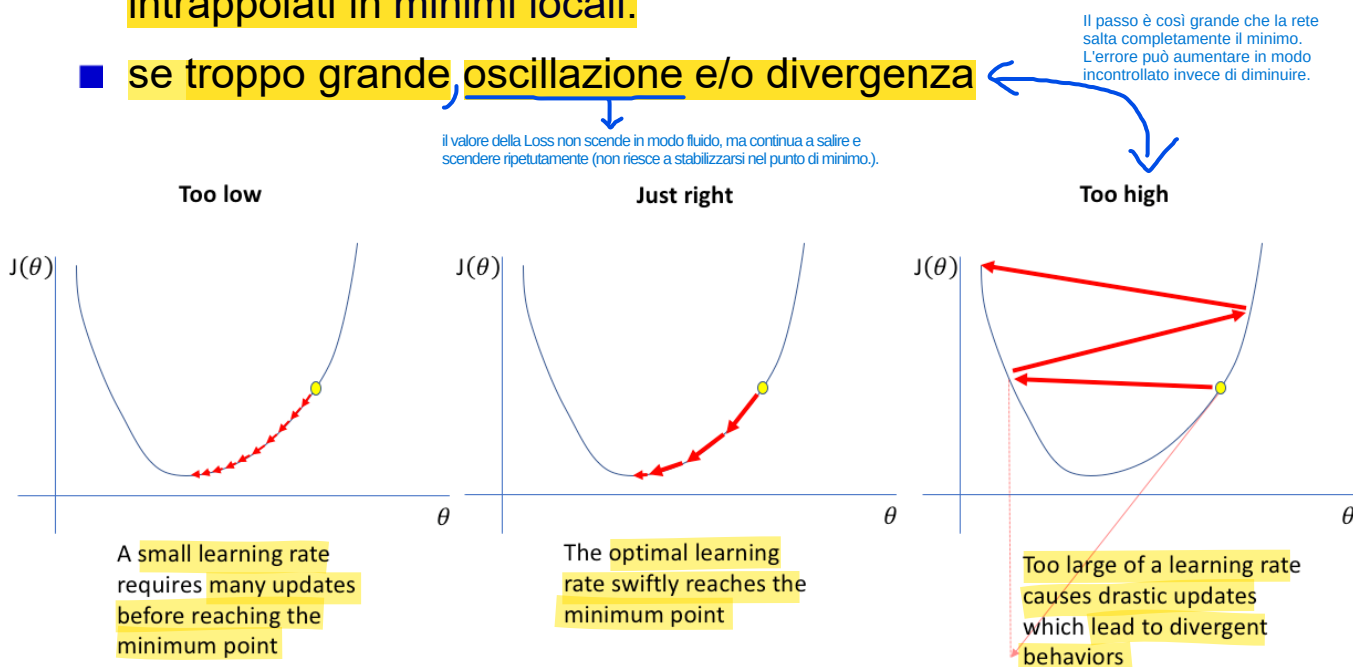
Analogo ragionamento per la regolarizzazione L1. Quanto vale la derivata del valore assoluto?

Tarare il Learning Rate

Ottimizzare

■ **Tarare il learning rate η in modo ottimale è molto importante:**

- se **troppo piccolo**, convergenza lenta ed è più facile rimanere intrappolati in minimi locali.
- se **troppo grande**, **oscillazione e/o divergenza**



■ il valore ottimale cambia a seconda dell'architettura della rete, della loss function, del minibatch size, ecc. **Consiglio: tararlo per ultimo, dopo aver definito tutto il resto.**

■ se si parte da architettura nota e già applicata da altri a problemi simili, partire dai valori consigliati ^{di learning rate} e provare a aumentare/diminuire monitorando convergenza e generalizzazione.

Questo assicura che il modello stia realmente imparando e non stia semplicemente convergendo lentamente verso un punto sub-ottimale.

■ **Attenzione:** il fatto che la loss diminuisca senza oscillare, non significa che il valore sia ottimale (specialmente nel tuning di reti pre-addestrate): **monitorare accuratezza sul validation set.**

usare scheduling learning rate: partire con learning rate alto e poi quando siamo vicini al minimo diminuire il learning rate

varie tecniche

■ Tecniche di **scheduling** di learning rate e/o ottimizzatori con learning rate **adattivo** possono essere di aiuto.

Scheduling del Learning Rate: Tecniche che modificano η nel tempo (es. ridurlo dopo un certo numero di epoche o quando la loss smette di migliorare).
Ottimizzatori Adattivi: Algoritmi avanzati (come Adam, RMSprop, Adagrad) che calcolano un Learning Rate specifico e dinamico per ciascun peso della rete.

Il Momentum è una tecnica che mira a risolvere due problemi principali del SGD standard con mini-batch:
 - Oscillazioni a Zig-Zag: Quando il paesaggio degli errori è ripido in una direzione e piatto in un'altra, il SGD tende a oscillare lungo la pendenza ripida, rallentando la convergenza verso il minimo.
 - Minimi Locali: Il SGD può rimanere intrappolato in minimi locali poco profondi.

Momentum

- Smorzamento delle Oscillazioni: Se il gradiente attuale sta spingendo il peso in una direzione opposta a quella media (oscillazione), l'inerzia compensa, smorzando l'oscillazione a zig-zag e permettendo passi più grandi.
 - Velocità Aumentata: Se i gradienti puntano costantemente nella stessa direzione, il termine di inerzia accumula forza, permettendo al processo di accelerare e di convergere più velocemente verso il minimo..

- SGD con mini-batch può determinare una discesa del gradiente a **zig-zag** che rallenta la convergenza e la rende meno stabile.

- la discesa del gradiente può essere vista come la traiettoria di una biglia che rotola su una superficie scoscesa. Attratta dalla forza di gravità la biglia cerca di portarsi sul punto più basso. Nella fisica il momento conferisce alla biglia un moto privo di oscillazioni e cambi di direzione repentini. Inoltre l'inerzia accumulata consente di superare piccoli avvallamenti (minimi locali).

se prende una curva a 100 km/h non sterza di 90 gradi
 cioè non fa un cambio repentino e spigoloso, ma inizia a curvare piano piano nella direzione della curva

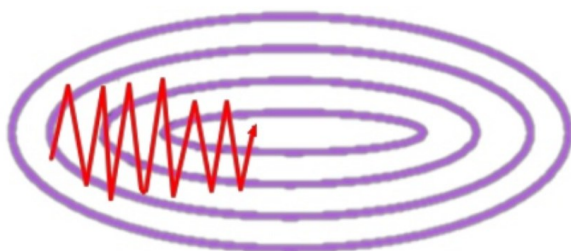
- in SGD per evitare oscillazioni si può emulare il comportamento fisico: si tiene traccia dell'ultimo aggiornamento Δw_k di ogni parametro w_k , e il **nuovo aggiornamento** è calcolato come **combinazione lineare** del precedente aggiornamento (che conferisce stabilità) e del gradiente attuale $\frac{\partial J}{\partial w_i}$ (che corregge la direzione):

$$\Delta w_k = \underbrace{\mu \cdot \Delta w_k}_{\text{Stabilità}} - \underbrace{\eta \frac{\partial J_{Tot}}{\partial w_k}}_{\text{Corregge Direzione}}$$

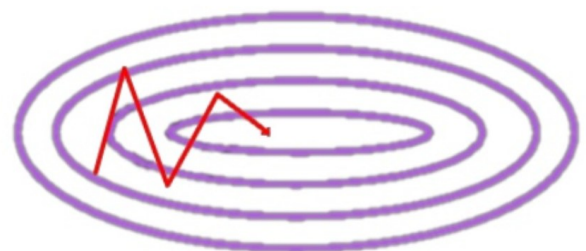
continuità rispetto a quello che ho appena fatto precedentemente
 quello che ho fatto al passo precedente

Rispetto allo pseudo-codice di SGD viene modificato solo la parte di aggiornamento dei pesi, se si applica SGD with Momentum

E il peso viene aggiornato:
 $w_k = w_k + \text{delta_}w_k$



Steps without Momentum



Steps with Momentum

Coefficiente di Momentum

Valore tipico del parametro $\mu = 0.9$

Learning Rate adattativo

■ Oltre a Momentum, esistono altri ottimizzatori (optimizers) per l'aggiornamento dei pesi che possono accelerare la convergenza nella discesa del gradiente (vedi Faster Optimizers in [A. Géron]):

- Nesterov Accelerate Gradient (NAG)
- Adaptive Gradient (Adagrad)
- Adadelta
- RMSProp
- Adam (e AdamW)

■ La maggior parte di queste adatta il learning rate η a ogni specifico parametro: passi di discesa più lunghi lungo le direzioni (parametri) in cui il gradiente varia poco. Necessario mantenere statistiche (parametro per parametro) sulla variazione gradiente. Adam è considerato lo stato dell'arte, anche se non sempre fornisce risultati migliori del più classico SGD with Momentum.

Gli ottimizzatori adattivi
- Principio Base: Questi metodi adattano il Learning Rate in modo dinamico, calcolando un tasso di apprendimento specifico per ogni singolo parametro (w_k).
- Logica: L'obiettivo è compiere passi di discesa più lunghi lungo le direzioni (cioè, per i parametri) in cui il gradiente varia poco (superfici piatte) e passi più corti lungo le direzioni in cui il gradiente è molto variabile (superfici ripide o rumorose).
- Meccanismo: Per fare ciò, questi ottimizzatori devono mantenere statistiche (storico) sulla variazione del gradiente per ciascun parametro, w_k .

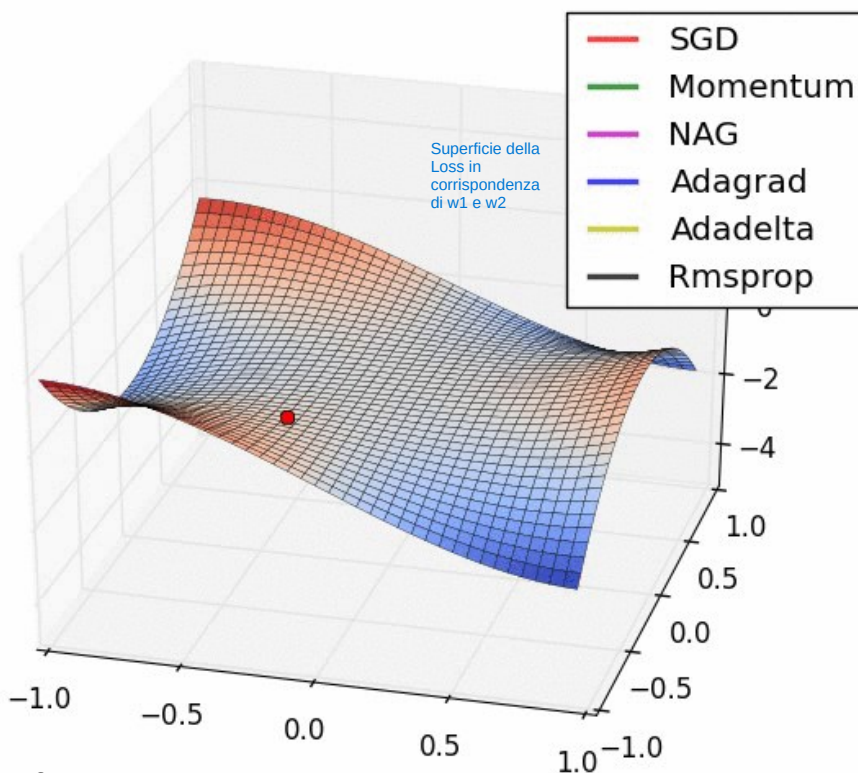


Image by Alec Radford

Il concetto che la rete non converge significa che, durante il processo di addestramento, l'algoritmo di ottimizzazione (la Discesa del Gradiente o i suoi derivati) non riesce a trovare i pesi ottimali per minimizzare la funzione di perdita (Loss Function). La non-convergenza si manifesta in due modi principali e ha cause diverse, quasi sempre legate a un errore di configurazione degli iperparametri o dei dati:

- La Loss Resta Alta e Stabile (Convergenza Lenta): Il valore della Loss Function è inizialmente alto e, dopo molte epoche, non diminuisce o diminuisce in modo estremamente lento.

- La Loss Oscilla o Aumenta (Divergenza): Il valore della Loss Function aumenta, oscilla violentemente, o esplode (tende a NaN o Infinito).

La rete non converge: check list

La convergenza di una rete neurale è lo stato in cui l'algoritmo di ottimizzazione (solitamente la Discesa del Gradiente o una sua variante, come Adam) ha raggiunto un punto in cui la funzione di perdita (Loss Function) non può più essere migliorata significativamente. In termini semplici, significa che la rete ha imparato il meglio che può fare, dato il training set e l'architettura.

■ Nel seguito alcune cose da verificare quando la convergenza sul training set non ^{arriva} parte (la rete sembra bloccata).

- 1 ■ Gli input sono stati normalizzati (min-max, standard scaling)?
- 2 ■ I pesi sono stati inizializzati random in range ragionevoli (es. Xavier, He)? altrimenti potremmo essere in zona saturazione da cui non è possibile uscire.
- 3 ■ I pattern delle diverse classi sono mescolati random all'interno dei minibatch? Nei mini-batch gli esempi devono essere mescolati casualmente
Se i pattern di una classe sono tutti insieme, il gradiente sarà "sbilanciato", allora training instabile o lento.
- 4 ■ Le etichette sono nel formato giusto? (interi sparsi, one hot vectors, float per regressione, ecc.)
- 5 ■ La loss function è corretta e adeguata al tipo di label fornite (attenzione alla differenza tra label sparse e vettori one-hot)?
- 6 ■ La funzione di attivazione all'ultimo livello è corretta (sigmoid per binary classification, softmax per cross_entropy multiclasse, null per regressione)?
- 7 ■ Il learning rate è adeguato?